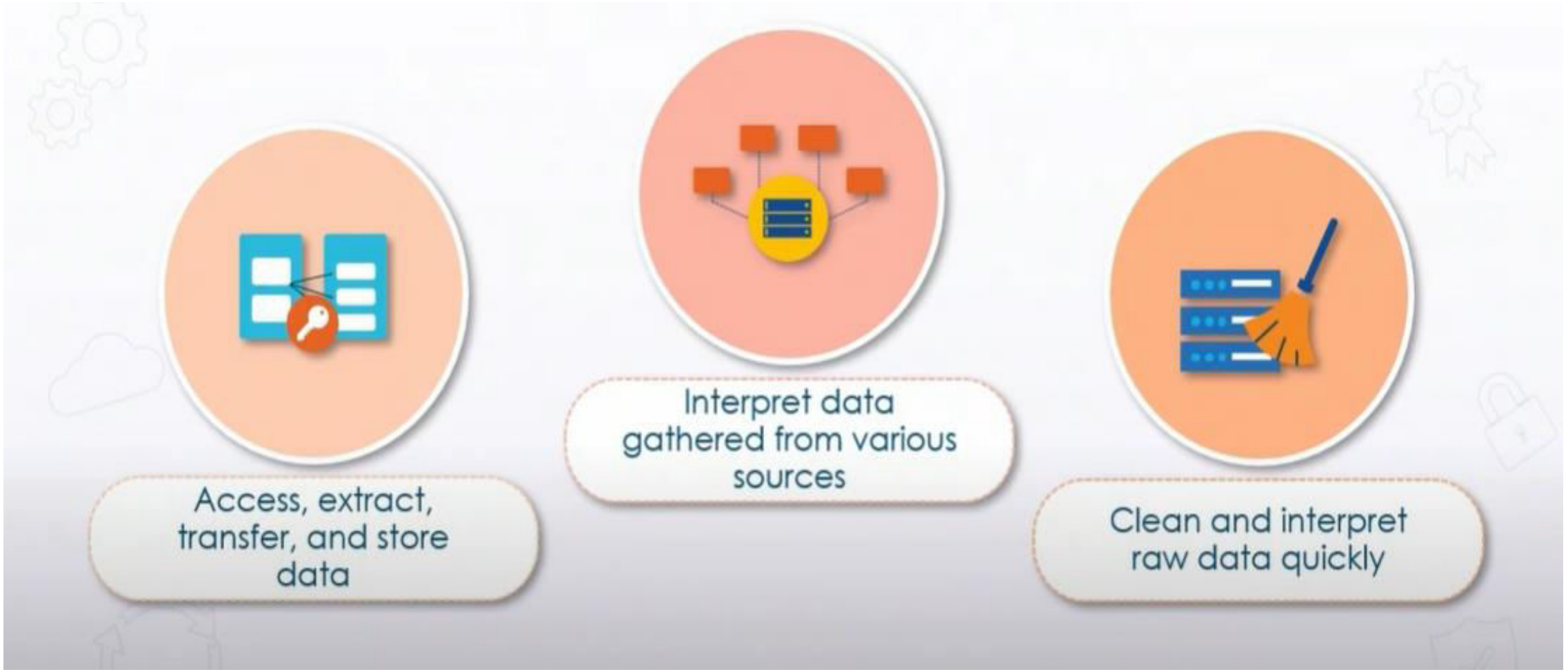
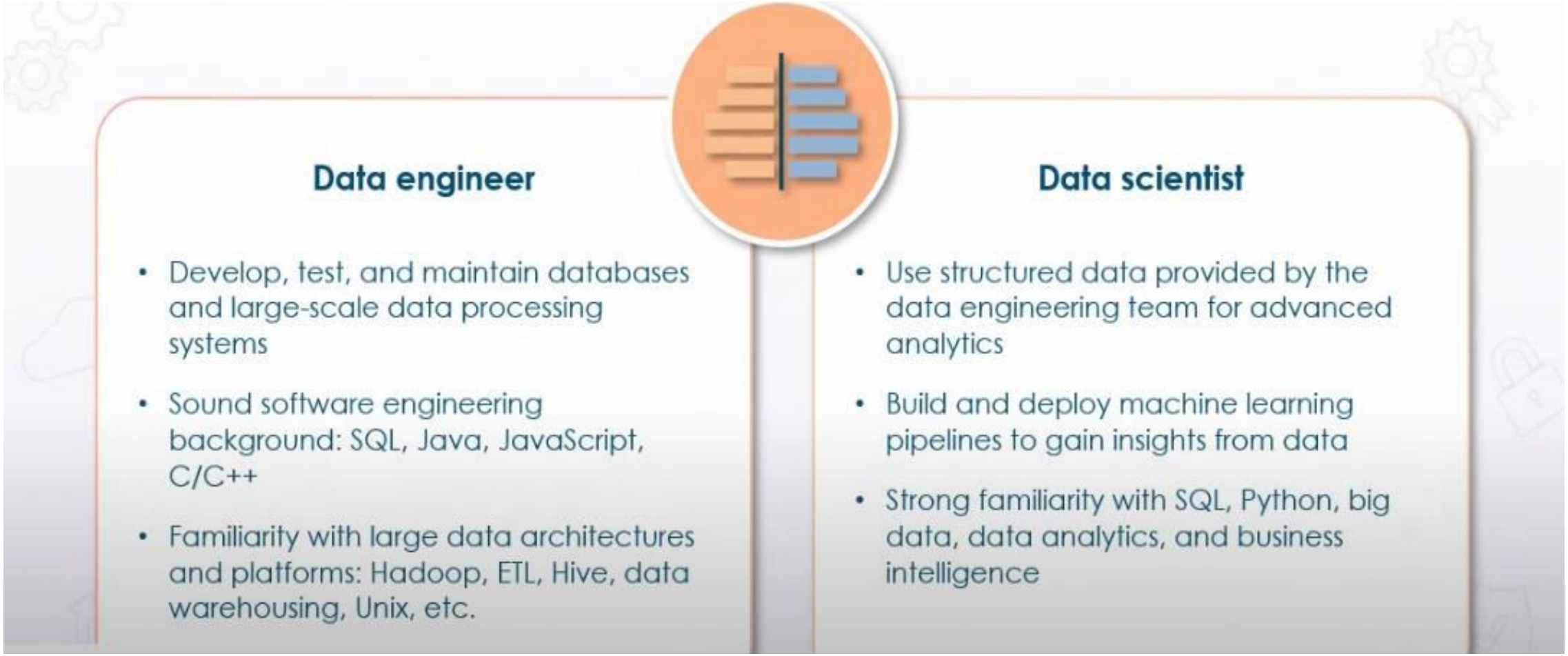
What is Data Engineering

The Data Engineer's Role



Data Engineer vs. Data Scientist Responsibilities



Data Engineering: An Evolution of Business Intelligence

Traditional business intelligence for large companies with large data volumes

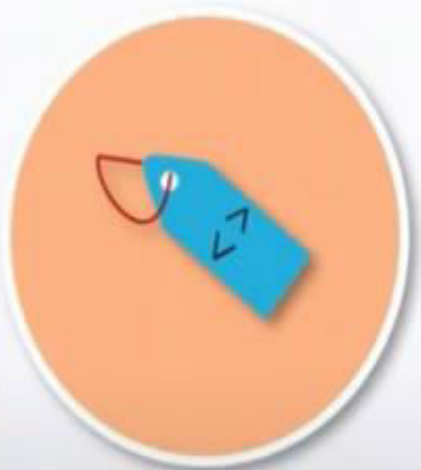
Advent of internet increased the amount of data exponentially

Large-scale cloud-based data solutions require data engineering as part of business intelligence

Real-world Data Engineering Applications

Data collection

Metric computation

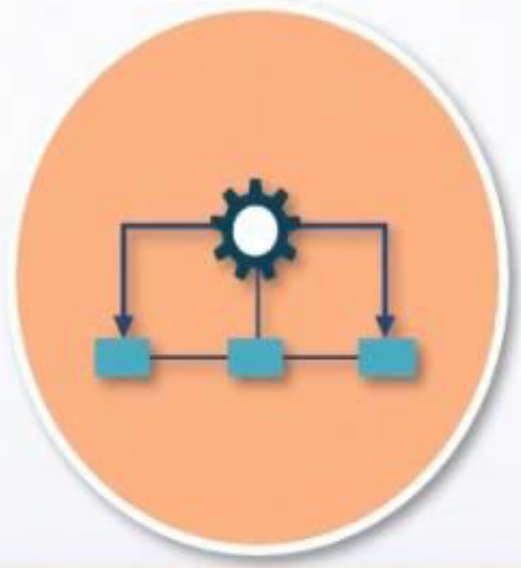


Metadata management

Data access tools

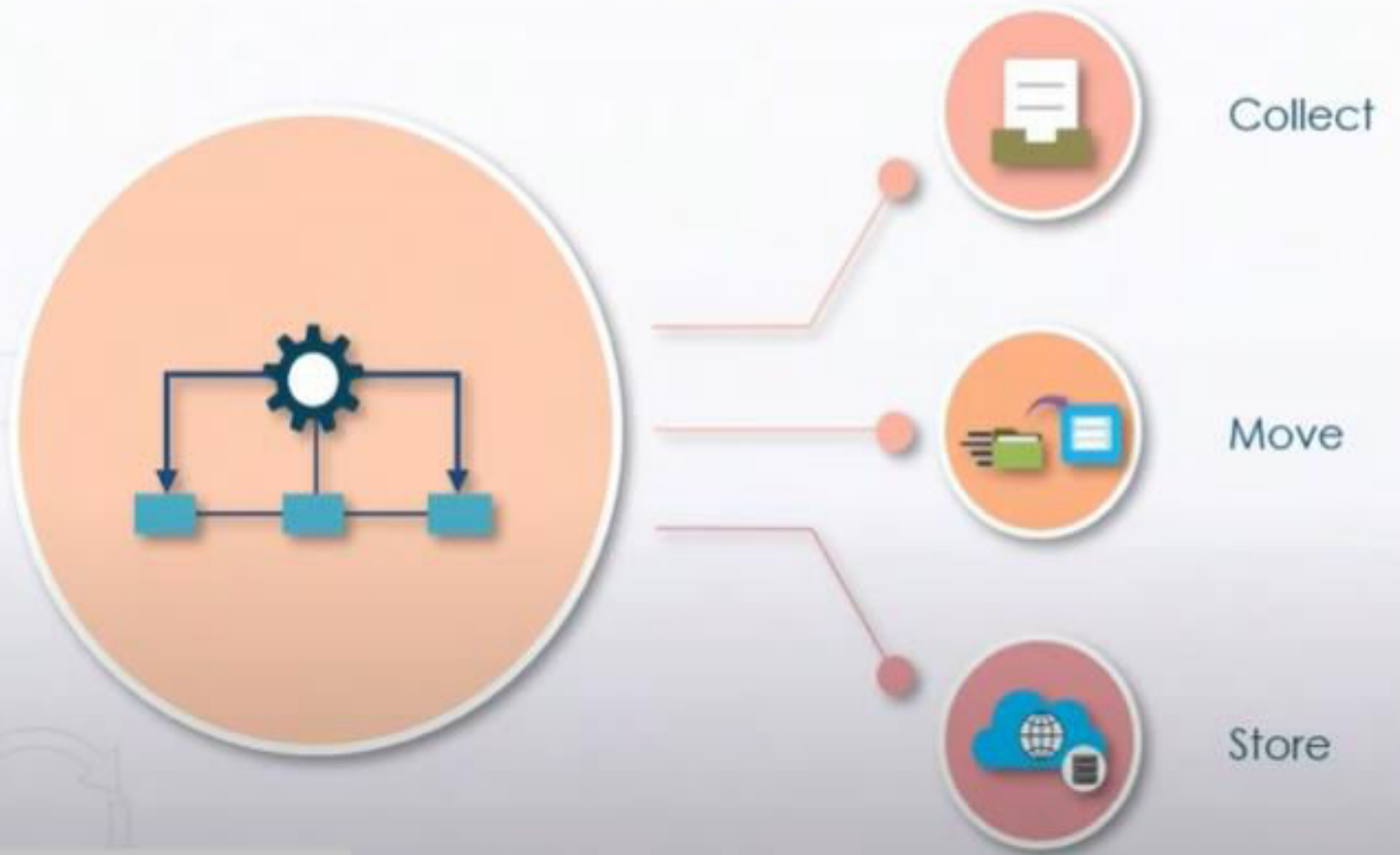
Hierarchy of Data Needs

Hierarchy of Data Needs

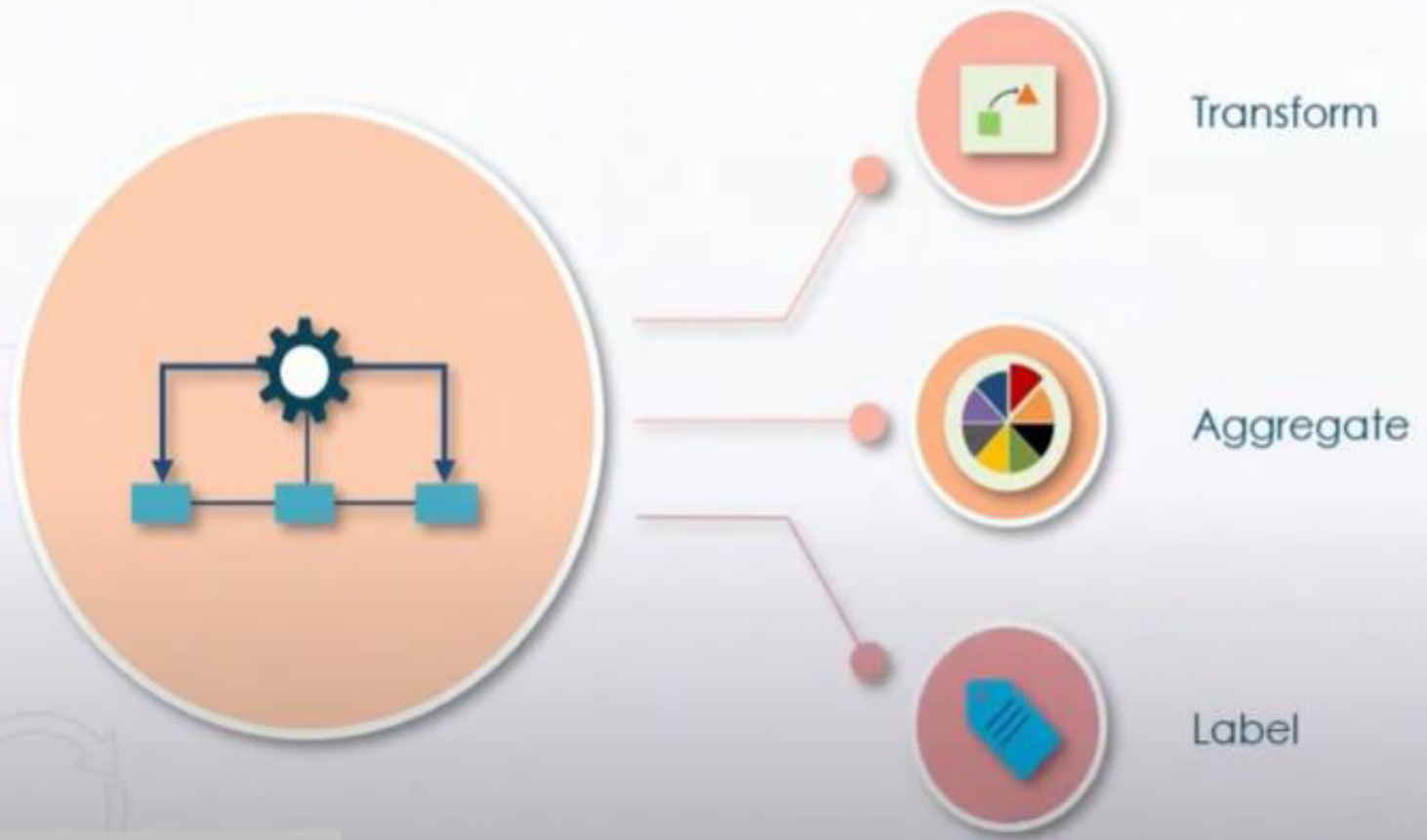


What is the proper sequential architecture to work with data?

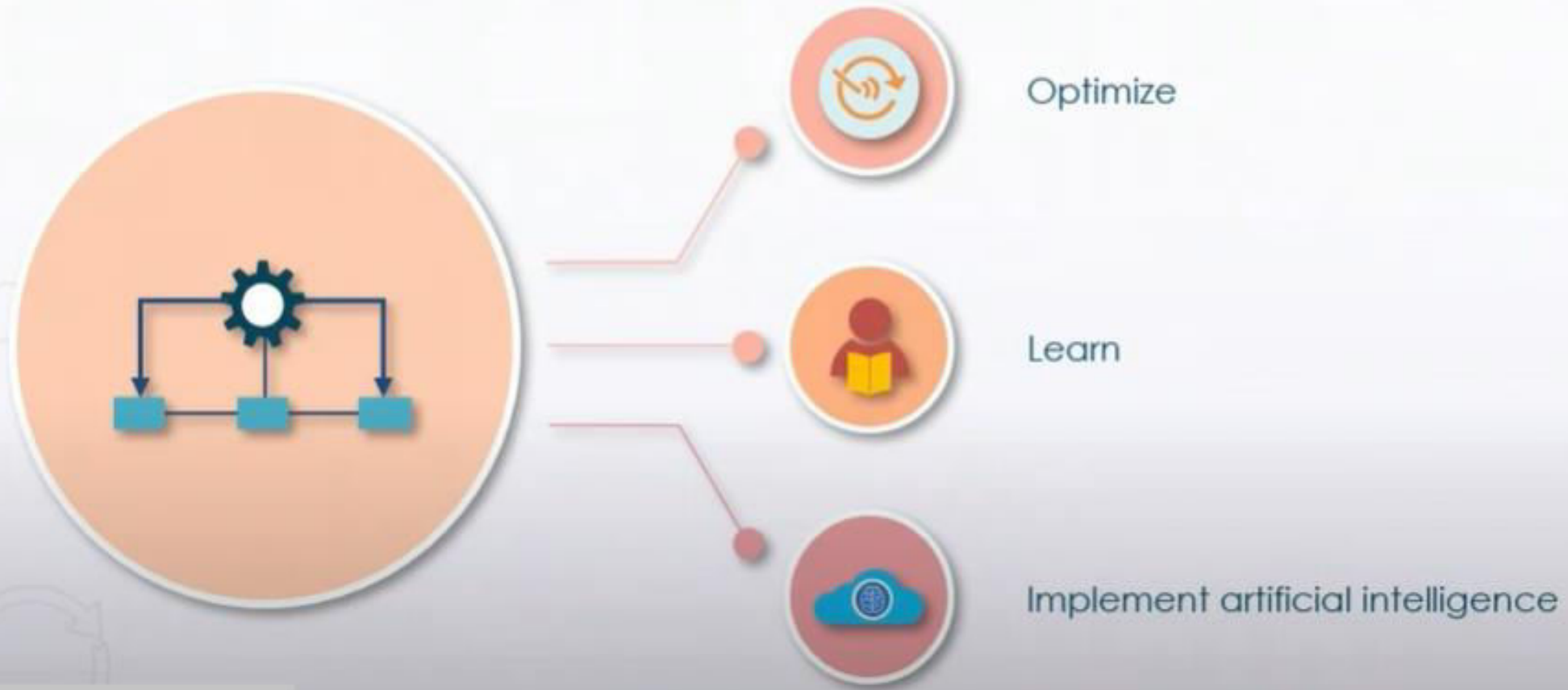
Collect, Move, and Store Data



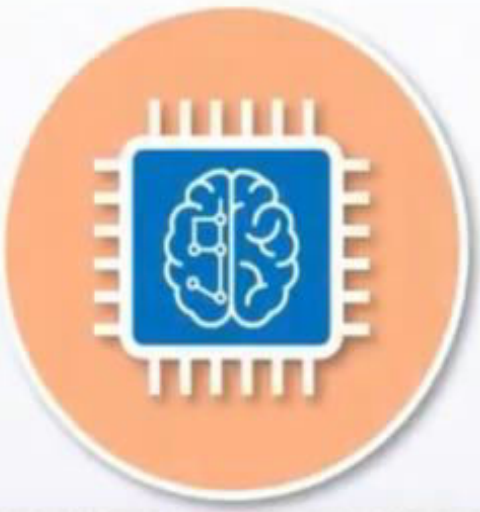
Transform, Aggregate, and Label Data



Optimize, Learn, and Implement AI

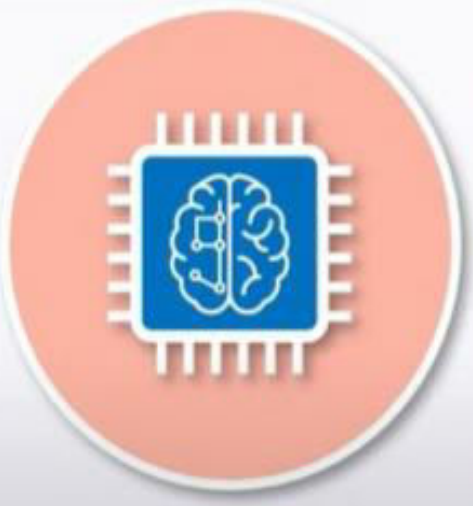


Machine Learning



What is machine learning?

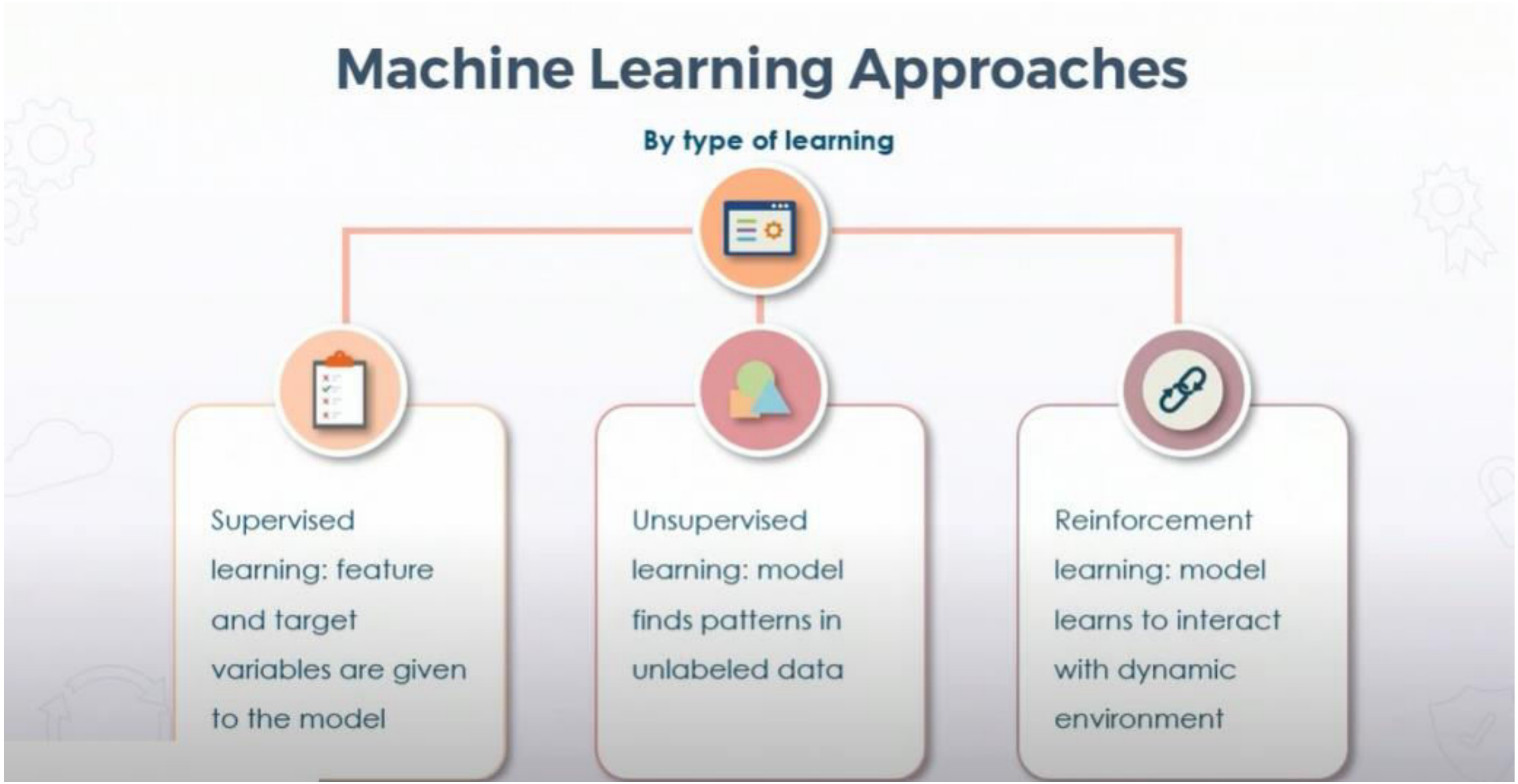
What Is Machine Learning Used for?



- Data analytics
- Predictive systems
- Suggesting products in real time
- Voice assistance
- Real-time map and route calculations

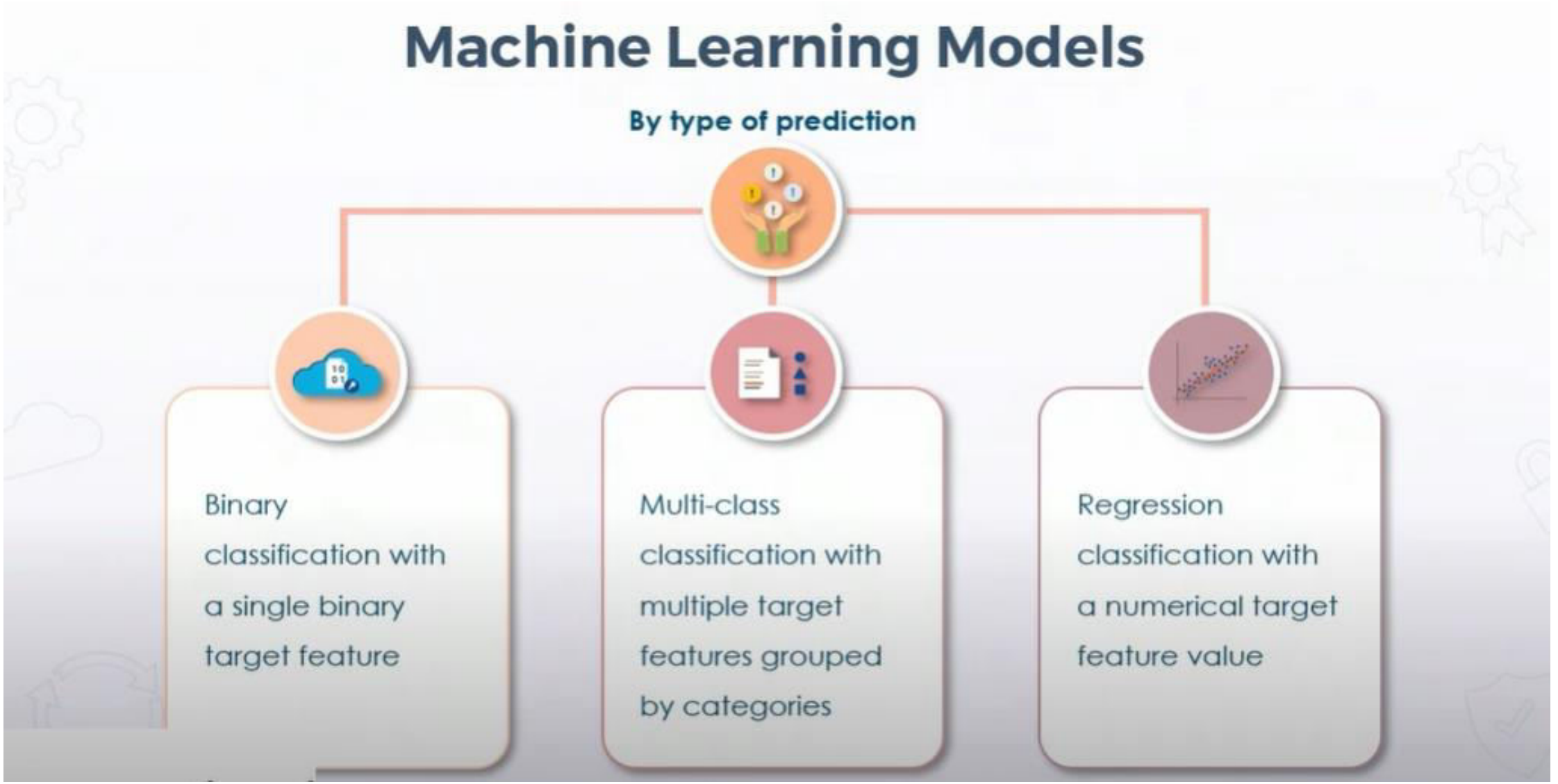
Machine Learning Pipeline

Machine Learning Approaches



Machine Learning Models

By type of prediction



Artificial Intelligence, Machine Learning, Data Mining, and Statistics

What is the difference?

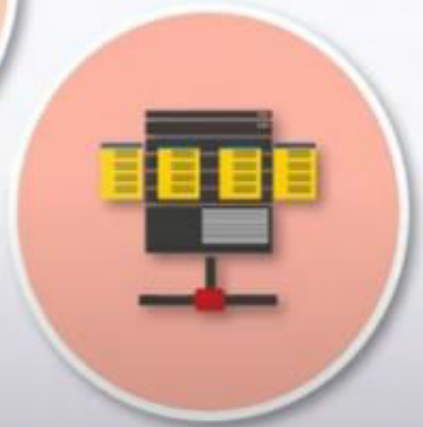
Machine Learning Limitations

Machine Learning Ethics

- Cultural and racial biases
- Financially impactful decisions
- Healthcare applications

Data Repositories and Data Warehouses

Data repository – large database infrastructure



Data warehouse – aggregated data repository

Data Warehouse Architecture and Organization

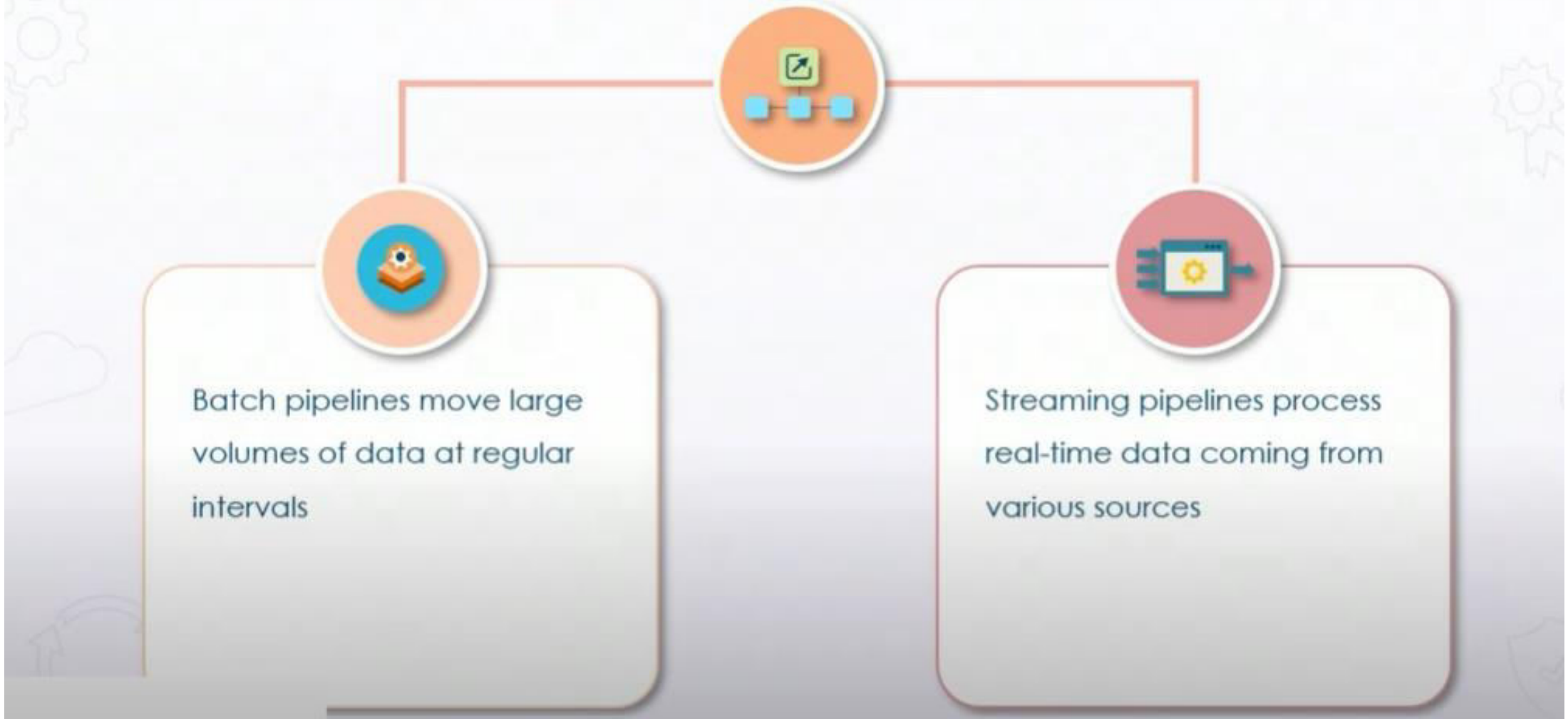
3-layered architecture:

- Front-end layer
- Analysis layer
- Database server layer

Data is organized in tables, which are connected using schemas

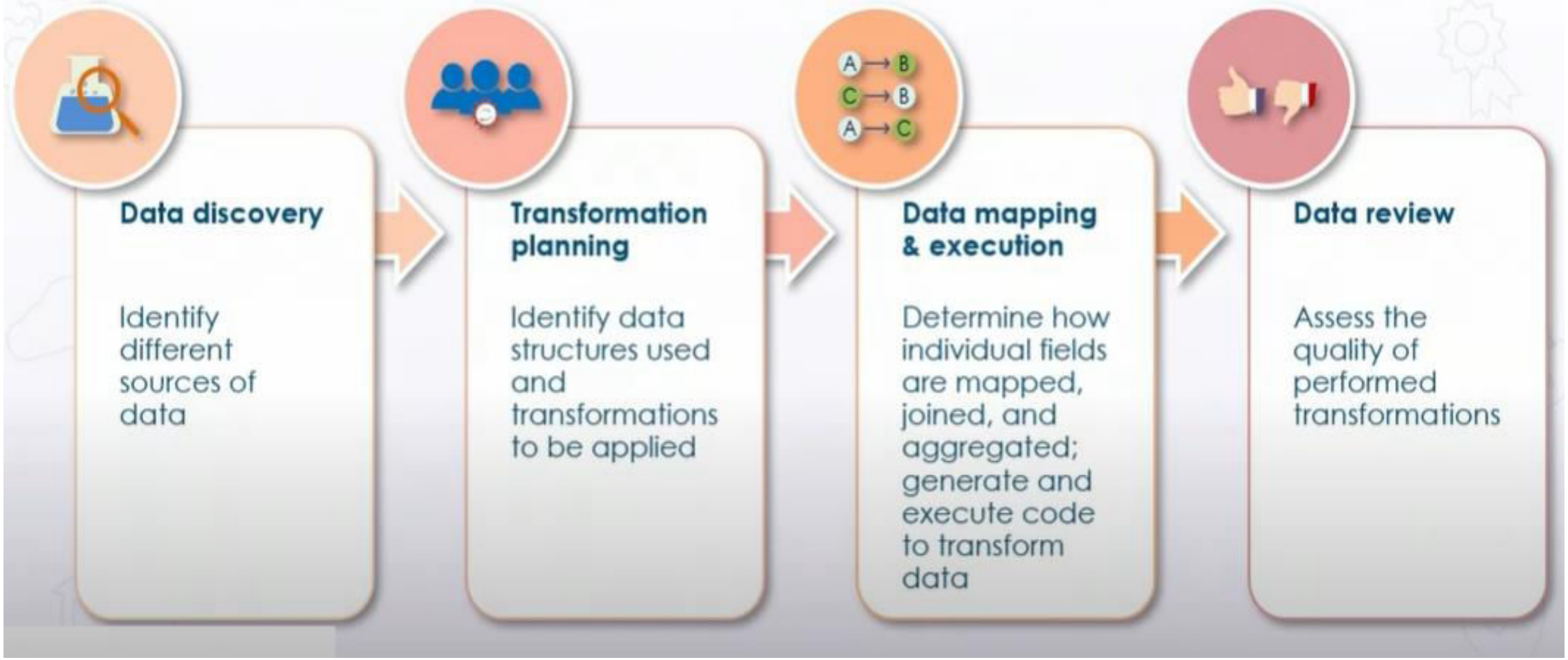
Data Pipelines
&
Data Ingestion

Types of Data Pipelines

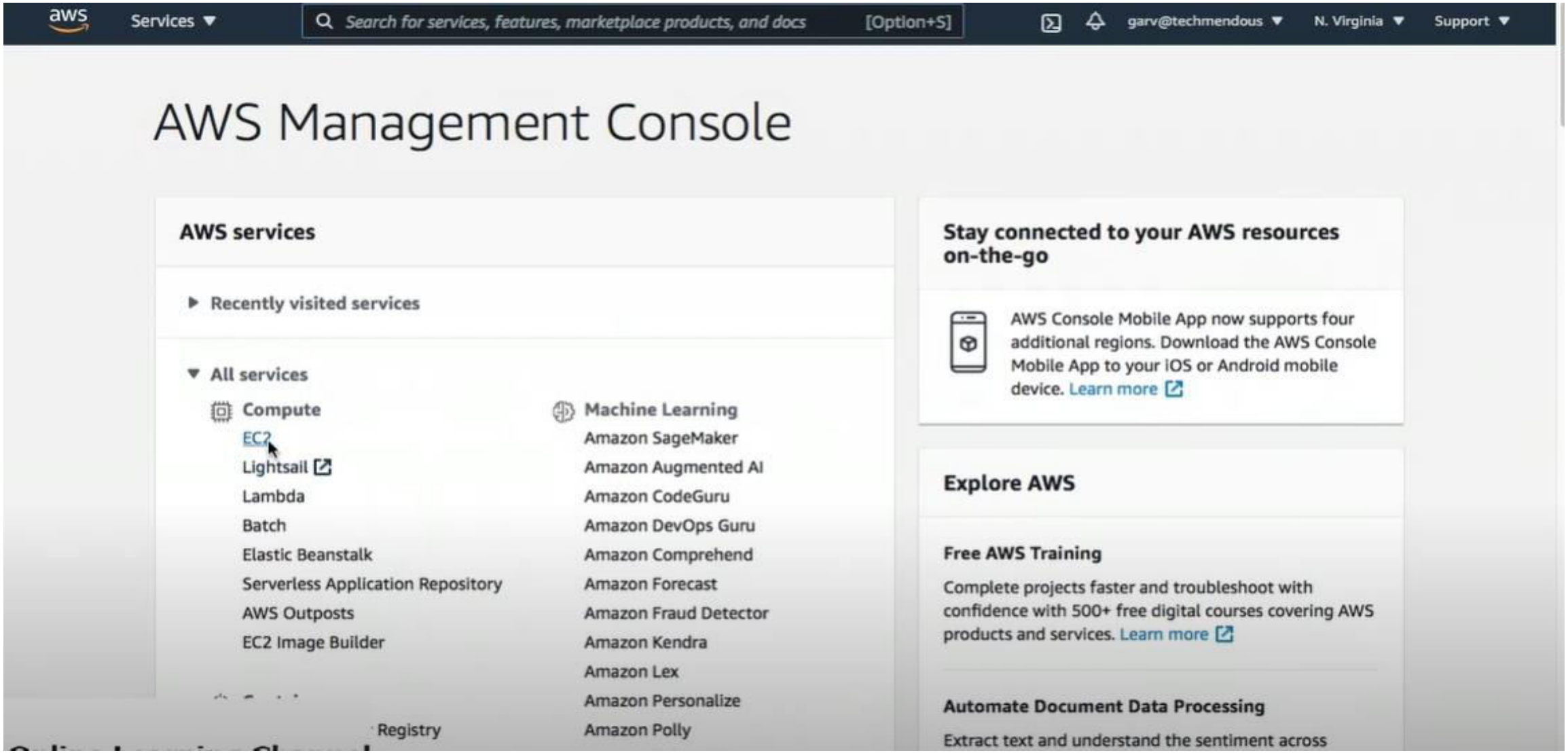


Data Ingestion Solutions

Data Transformation



What is AWS ?



https://youtu.be/WUAn8jCTAGU?si=C-yxsdYBsheztFnj&t=1487

Thank You

Amazon Simple Storage Service

What is Amazon S3?

Key Amazon S3 Benefits

Data Categorization in Amazon S3

Object-storage architecture

Amazon S3 Object Storage Architecture

Amazon S3 Storage Lens

Object storage usage and activity – single view tool

Amazon S3 Storage Lens Features

View overall insights summary as well as cost efficiency and data protection recommendations

Visualize usage trends and identify outliers

Identify fastest growing buckets and total storage across organization

Amazon S3 Intelligent Tiering

Optimization of storage cost

Amazon S3 Access Tiers

Amazon S3 Storage Classes

Standard Infrequent Access Storage class

Standard data storage class

One Zone-Infrequent Access Storage class

Amazon Glacier & Glacier Deep Archive

Amazon S3 Access Points

Simplify bucket access across multiple users

Amazon S3 Access Points

Limit public access to a bucket

Limit access to a VPC or a virtual private cloud

Amazon S3 Batch Operations

Amazon Data Management feature

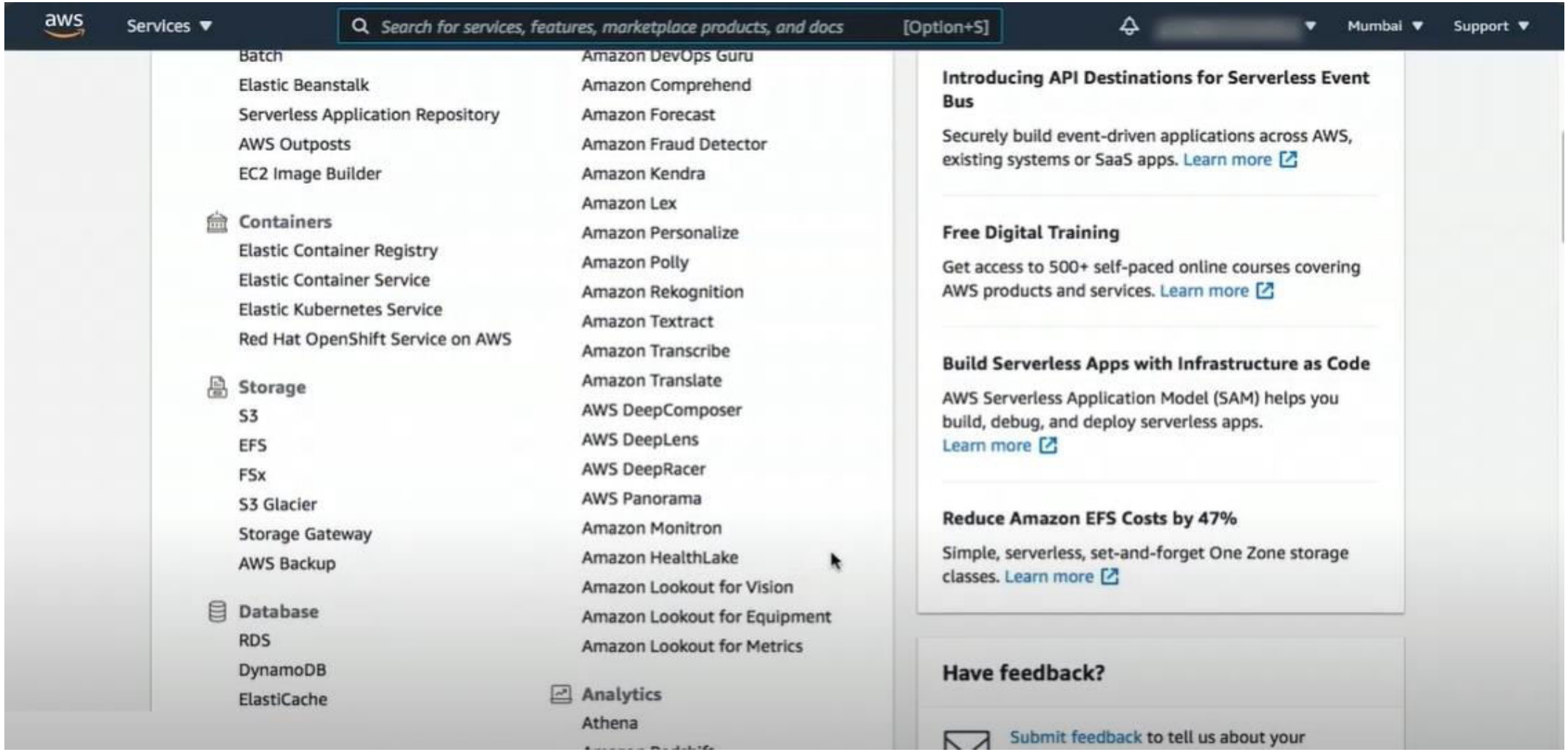
Amazon S3 Batch Operations

- Run multiple jobs and set precedents of jobs based on priority
- Track progress, send completion notifications, and create reports
- Select tasks to be performed through management console
- Use Lambda function to define complex tasks across millions of target objects

Amazon S3 Block Public Access

Amazon S3 Block Public Access

- Implement by few clicks within management console
- Block all public access except access control lists (ACLs) or bucket policies
- Additional layer of protection for accounts and buckets



https://youtu.be/ESbgV5IsUa4?si=ruFbgUdX0nUmdA3K&t=813

Amazon S3 Use Cases : Archives, Data Lakes, Analytics

Amazon S3 Use Cases

Amazon S3 Case Studies

Amazon S3 Case Studies

Netflix

Uses S3 for most computing and storage needs

Deploys thousands of servers with terabytes of data

GE Healthcare

Incoming medical data is stored in S3

Durable, secure, and reliable storage for critical data

Ryanair

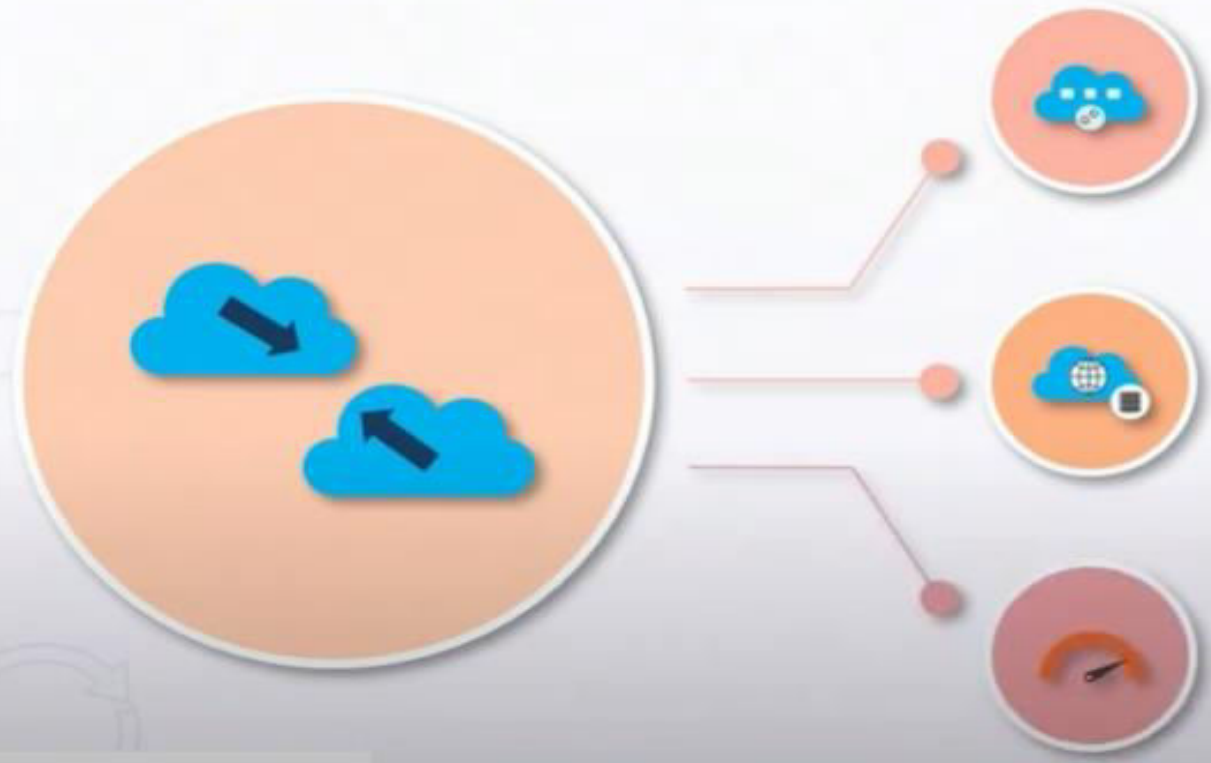
Uses S3 Glacier and Glacier Deep Archive for long-term storage

Saves 65% on backup costs annually

Thank You

AWS GLUE

What Is AWS Glue?



Amazon data integration service

Discover, clean, combine, and prepare data

Shorten the time needed to prepare and integrate data

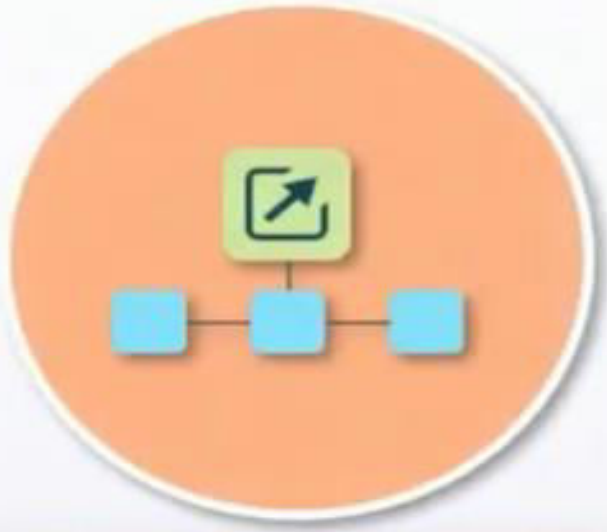
AWS Glue Advantages

- Automate data integration processes
- Suggest schemas for storing data
- Generate code for data transformation
- Create catalogue of metadata

AWS Glue Core Features

- Event-based triggers and parallel ETL jobs
- Built-in fault tolerance and event logs
- Serverless service requiring no infrastructure
- Combination and replication of data from various sources

AWS Glue



Extract, Transform, Load pipelines

AWS Glue ETL Pipelines

Event-based ETL pipeline

Central, unified catalogue

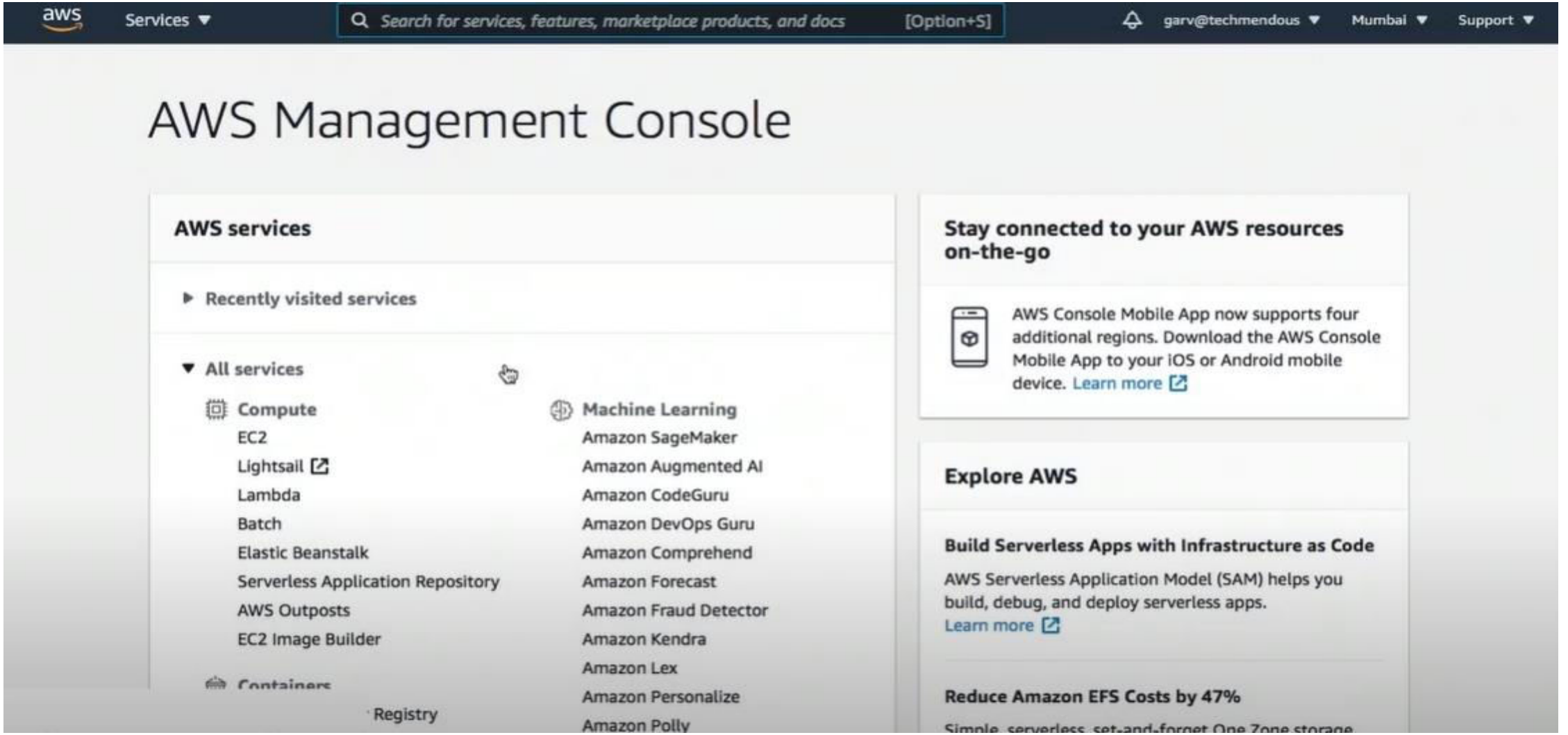
AWS Glue Features

AWS Glue Studio

- Manage ETL pipelines without writing code
- Move data from any data store
- Create complex ETL pipelines using a visual interface
- Send to other AWS services for deriving insights

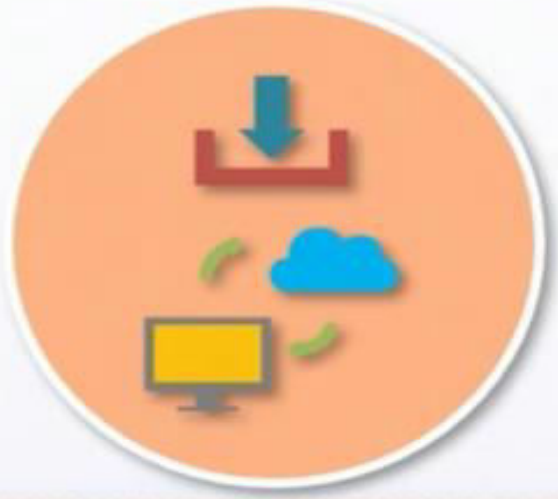
AWS DataBrew

- Visual data preparation tool
- Evaluate quality of the data
- Clean and normalize data using a visual interface
- Makes data preparation 80% faster



https://youtu.be/IaL6yfTxbSA?si=fE4o9ZNzLIqU4Dfb&t=302

What Is Streaming Data?



Data that is continuously generated by various sources

Streaming Data

- Processed sequentially
- Used to assess correlations and get insights into various business activities
- Helpful in making business-intelligence decisions promptly

Applications and Benefits of Streaming Data

Large amount of data sources

Process, analyze, and respond to data promptly

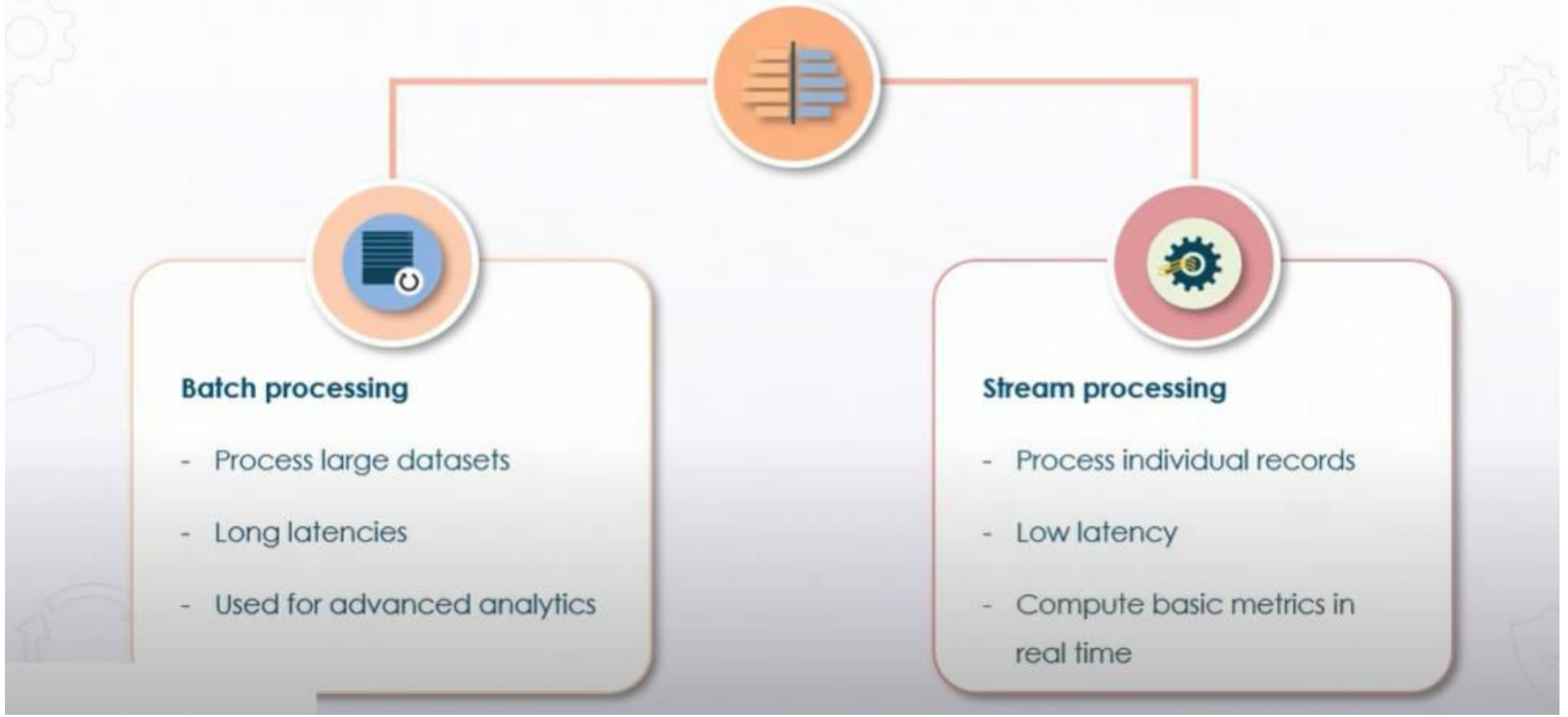
Real-time data analysis and simple reporting & tracking

Data Processing

Batch processing

Stream processing

Batch Processing vs. Stream Processing



Amazon Kinesis: Benefits and Use Cases

Amazon Kinesis

Collect, process, and analyze streaming data

Amazon Kinesis

- Real-time ingestion of streaming data
- Immediate business response
- Derives insights in several seconds
- Scales to manage large volumes of streaming data

Amazon Kinesis Capabilities

- Analyze video, audio, and data streams in real time
- Collect streaming data on AWS for analytics, machine learning, and processing
- Firehose captures, transforms, and loads streams directly to AWS data stores
- Analyze incoming streams using SQL or Apache Flink

Amazon Kinesis Use Cases

- Analyze home, office, or factory video streams
- Record and update public data in real time
- Zillow uses Amazon Kinesis to update home prices
- Sonos uses Amazon Kinesis to ingest billions of events from its wireless devices

Amazon Kinesis Video Streaming

- Ingest, securely store, and encrypt video streams
- Scale to ingest video streaming from millions of devices
- Build applications with real-time computer vision and machine learning
- Automatic encryption and peer-to-peer sharing support

Amazon Kinesis Video Streaming

Use case example

- Remote doorbell connected to a smartphone
- Amazon Kinesis streams the video from the doorbell to the smartphone and vice-versa
- This is used for real-time communication with near zero latency

AWS Kinesis: Data Streams

Amazon Kinesis Data Stream

Capture large amounts of data

Amazon Kinesis Data Stream

Use case example

- Gaming platform that records player interactions
- Data is continuously fed to the gaming platform and analyzed in real time
- The game dynamically reacts to user actions, making the interaction experience more engaging

Amazon Kinesis Data Firehose

- Transforms and delivers streaming data to AWS services
- Scales automatically based on amount of incoming data
- Works in near real-time speed continuously

Amazon Kinesis Data Firehose Key Difference

- Can be scaled to handle gigabytes of streaming data per second
- Supports batch processing, encryption, and compression of data
- Provides automatic scaling to meet demand

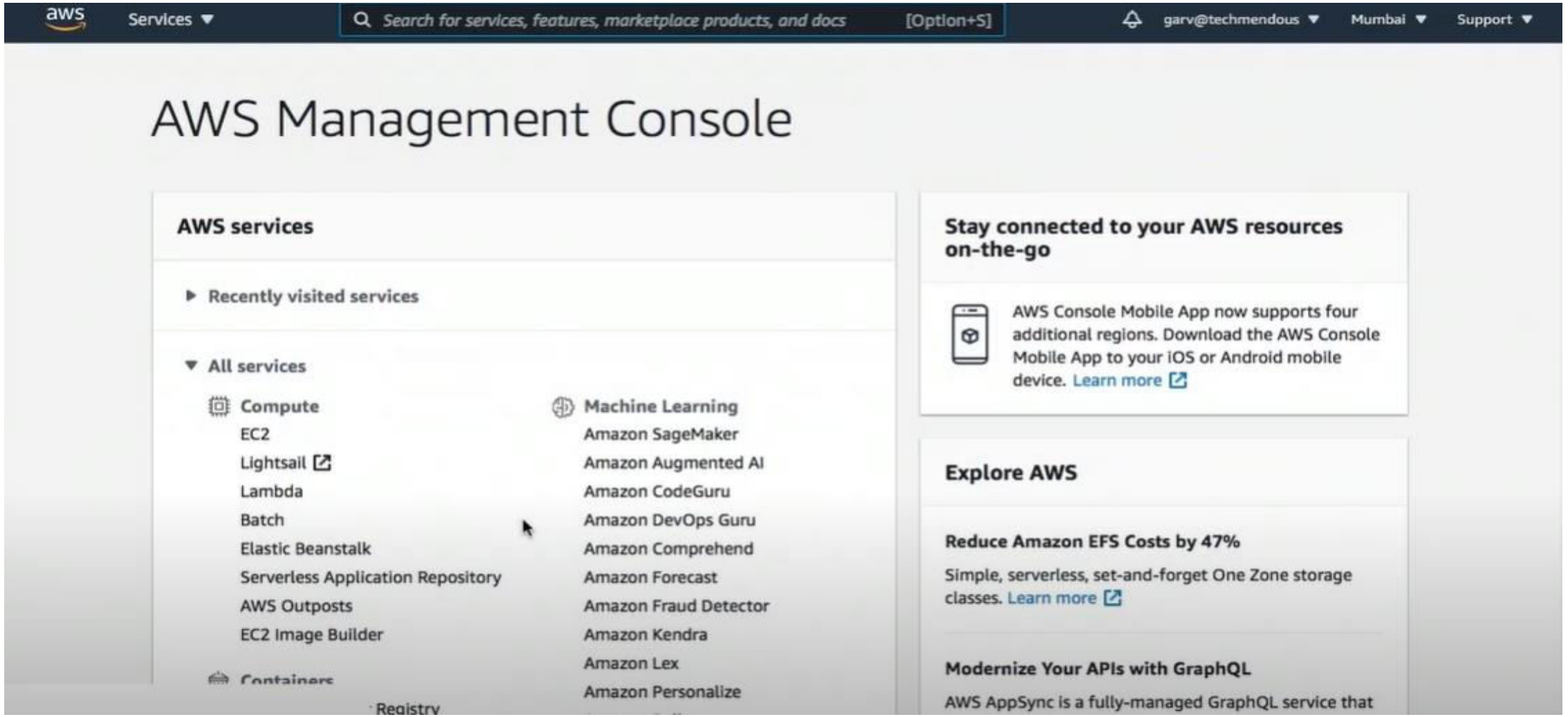
AWS Kinesis: Data Analytics

Amazon Kinesis Data Analytics

- Integrate Apache Flink with other AWS services
- Scales automatically to meet demand
- Capability to filter, aggregate, and transform data for advanced analytics

Amazon Kinesis Data Analytics Use Case

Gunosy processes half a million records per minute creating personalized and curated news feeds using Amazon Kinesis Data Analytics



https://youtu.be/IaL6yfTxbSA?si=ENKOKUN1frjZao5Y&t=1517

Thank You

What Is AWS Data Pipeline?

AWS Data Pipeline Overview

Enable movement and processing of data between AWS services

Access, transform, and send data to desired storage service

Manage workload with fault-tolerant and easy-to-use console interface

AWS Data Pipeline Benefits

- Automated data movement tasks
- Easy inter-task dependency management
- Highly scalable service
- Supports serial and parallel data interactions
- Available at low cost through user optimization

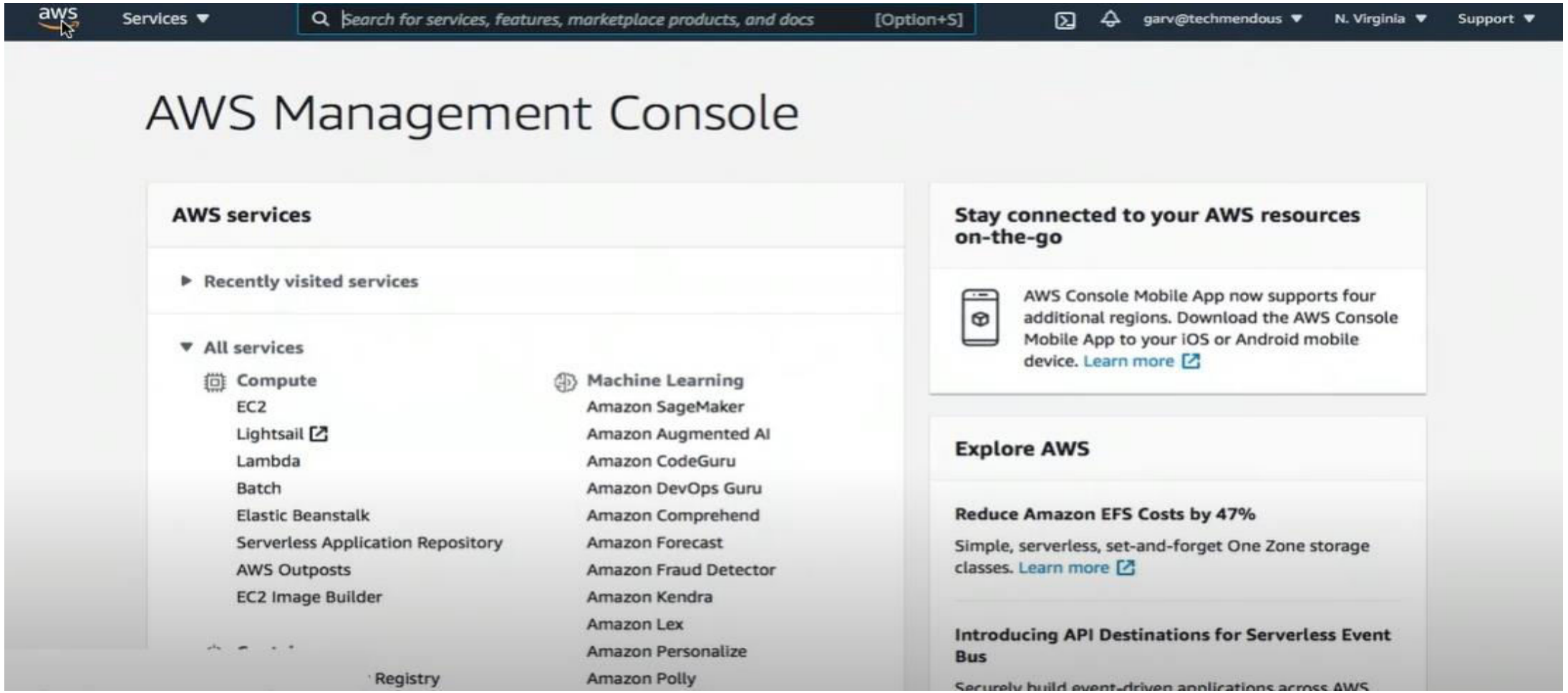
Getting Started with AWS Data Pipeline

- Create pipeline definition file
- Specify names, locations, and formats of data sources
- Create definition file using AWS console or as a JSON file

AWS Data Pipeline Use Cases

- Combine structured and unstructured data
- Copy, transform, and analyze data across DynamoDB and Amazon S3
- Data movement between RDS and/or Redshift and Amazon S3
- Back up DynamoDB tables to Amazon S3

Configuring AWS Data Pipeline



https://youtu.be/s_wtYi_TppA?si=QrtWdIQmn9hCV9Vk&t=210

AWS Data Pipeline vs. AWS Glue

Key differences

AWS Data Pipeline vs. AWS Glue

AWS Data Pipeline

Service that helps users transfer data on the AWS platform by:

- Defining
- Scheduling
- Automating
- Monitoring from single location

AWS Glue

Big data cataloging tool that allows users to perform ETL tasks on the AWS platform

AWS Data Pipeline vs. AWS Glue

Pricing:

- AWS Data Pipeline: 1$ per month per pipeline
- AWS Glue: 44c per data processing hour plus additional fees

Use cases:

- AWS Glue: transform data and store it in target destination
- AWS Data Pipeline: transform and move data across various sources

Infrastructure:

- AWS Glue: serverless service
- AWS Data Pipeline: not serverless; allows more control of resources

Operational methods:

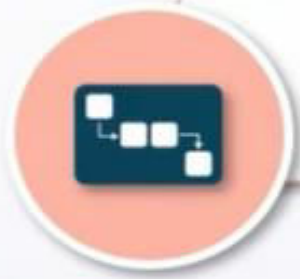
- AWS Data Pipeline: create data transformations through APIs and JSON
- AWS Glue: additional support of Apache Spark, Scala, and Python

AWS Batch Overview

- Build efficient and long-running computational jobs
- Handy when users need to run multiple jobs
- Organized into four different components including jobs, job definitions, job queues, and compute environment

AWS Batch Overview

Job unit – work that is submitted to AWS Batch



Job queue – list of work that needs to be completed

Job definition – description of how the work has to be executed

Compute environment – computational resources that run a job

AWS Batch Benefits

- No installations are required
- Simple and scalable solutions to manage and monitor computing requirements
- Complete integration with AWS platform

AWS Batch

Use cases

AWS Batch Use Cases

Financial services

- End-of-day trade processing
- Minimization of human error
- Cost reduction through automation
- For pricing, market position, and risk management workloads
- Trade desk performance improvements
- To apply analytics & machine learning algorithms to financial transaction data

Life sciences

- For drug screening
- To discover and identify structures that bind to a target drug
- To apply compound and complex screening jobs to better understand biochemical processes

What Is AWS Step Functions?

AWS Step Functions – Workflow Editor

Execute tasks automatically

Trigger and track workflow steps

Ensure correct order of execution and handle errors

AWS Step Functions Benefits

- Invoke Lambda functions on AWS jobs
- Fetch or insert items from storage databases
- Quickly create complex sequence of tasks
- Build applications without writing code
- Set up queues and databases for communication between serverless services

AWS Step Functions – Usage

- Limit of 25,000 transitions per execution and 1MB payload limit
- AWS free tier includes 4,000 AWS Step Functions state transitions per month
- Additional transitions priced at 0.025$ per 1,000 transitions

AWS Step Functions Use Cases

AWS Step Functions – Use Cases

Create machine learning workflows

- Coordinate building, training, and deployment of machine learning algorithms
- Automate data preprocessing using AWS Glue
- Preprocessed data is sent to Amazon SageMaker to train machine learning models
- After model is trained, it can be automatically deployed to make predictions

AWS Step Functions – Use Cases

Orchestrate multiple ETL jobs

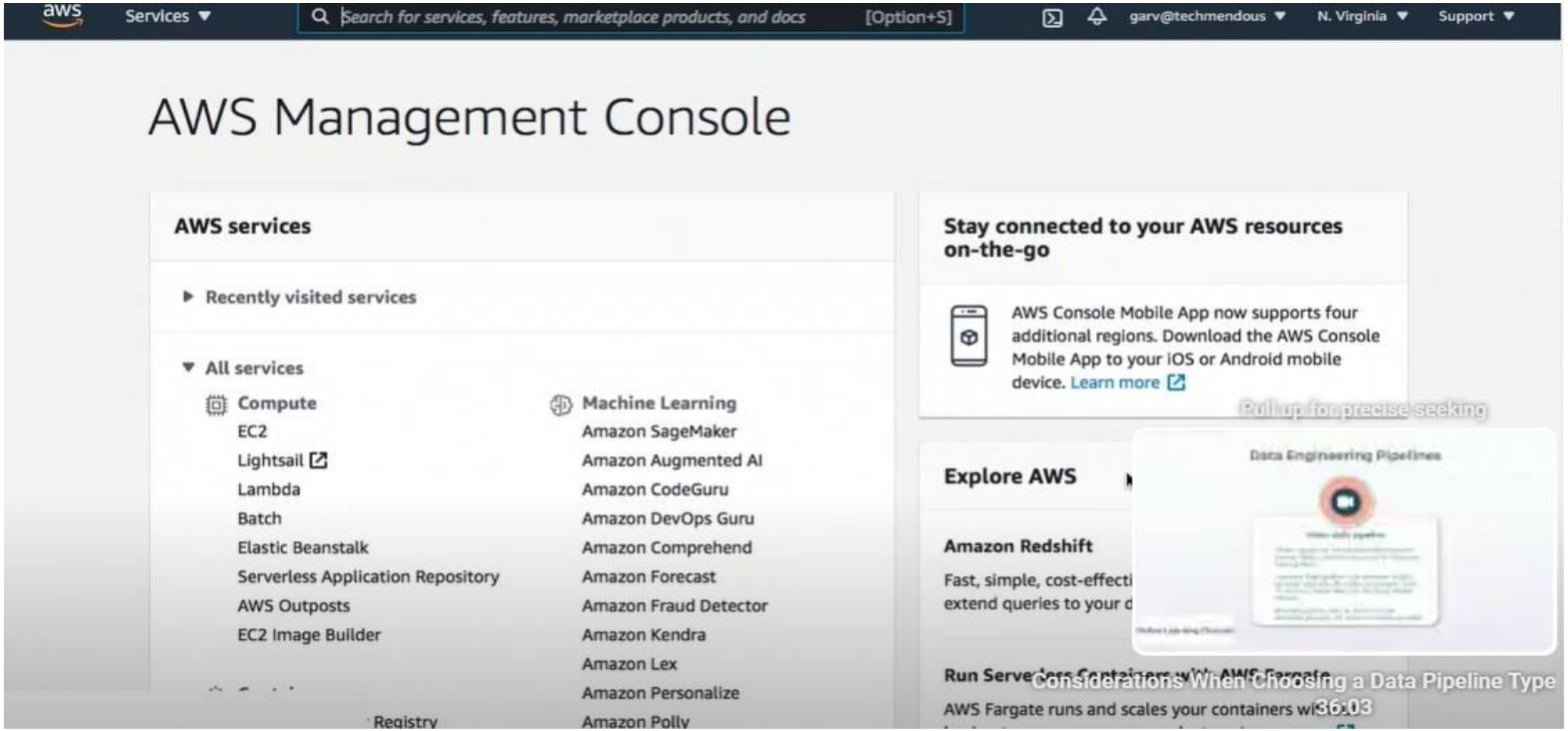
- Connect large set of different technologies in a complicated ETL workflow
- Coordinate multiple jobs on AWS Glue

AWS Step Functions – Use Cases

Media transcoding

- Coordinate tasks for image analysis
- Data tasks can be triggered immediately upon uploading an image to Amazon S3
- Possible tasks include

  1. creating thumbnail images
  2. extracting metadata, and
  3. running Amazon Rekognition object detection pipelines

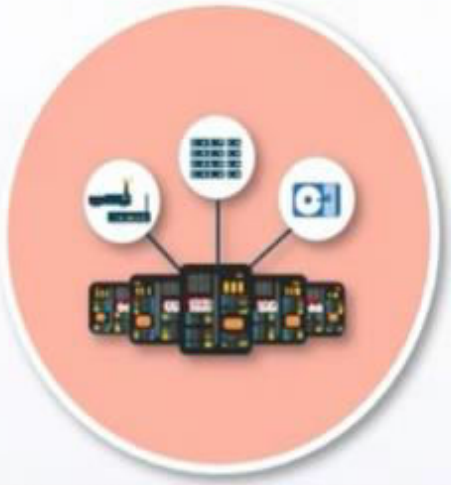


https://youtu.be/s_wtYi_TppA?si=luxRoX6b4wcdkb5A&t=1568

Real-time and Video Data Pipelines

Considerations When Choosing a Data Pipeline Type

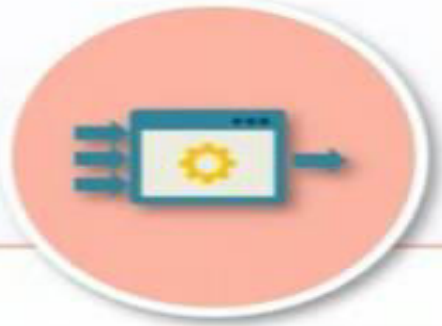
What is the reason to create the pipeline?



What type of data will be processed?

What will the data be used for?

Data Engineering Pipelines



Real-time data pipeline

- Consists of data stream origin, parsing and filtering instructions, data transformation, and loading of resulting data into destination data lake
- Can be collected from various sources, including logs, mobile devices, sensors, and Amazon Kinesis data streams
- The processed data can be analyzed immediately using Amazon services

Data Engineering Pipelines

Video data pipeline

- Video data can be loaded into Amazon Kinesis Video Streams and sent to Amazon Rekognition
- Amazon Rekognition can perform video analysis and identify objects, people, text, or scenes, generating an Amazon Kinesis stream
- Resulting data can be sent to other Amazon services for more complex analysis

Batch Processing and Analytics Pipelines

Data Engineering Pipelines

Batch processing

- Manage and monitor batch processing jobs
- Scale computational requirements based on immediate needs
- Automate and configure various AWS resources required to manage complex batch processing pipelines

Data Engineering Pipeline

Analytics layer

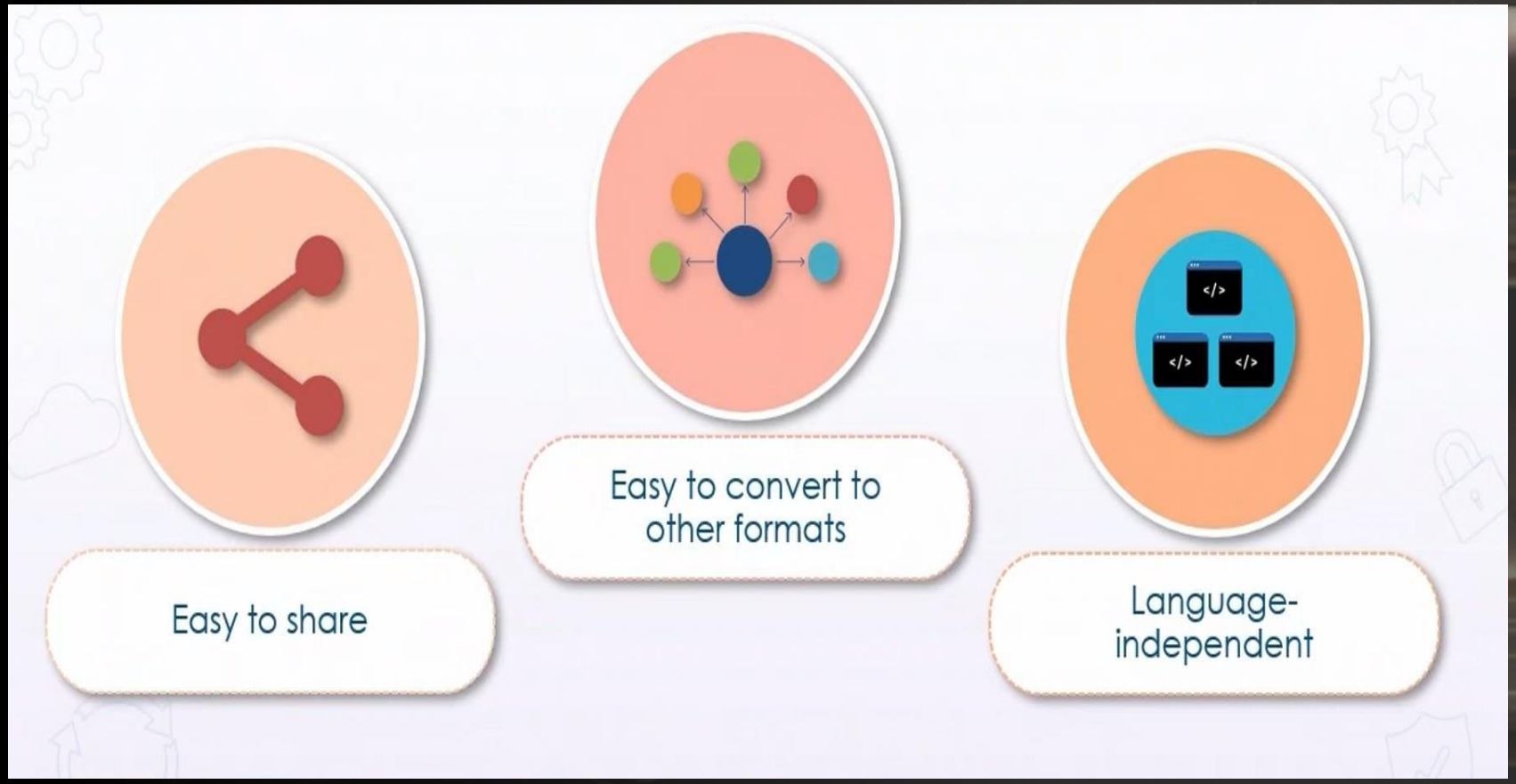
- Analyze AWS warehouse data using Amazon EMR – tool for processing and analyzing big data
- Take advantage of open-source tools like Spark, Hive, Flink, and Presto
- Amazon Redshift is a cloud data warehouse that allows combining structured and semi-structured data using standard SQL
- Pass the results of queries to other Amazon services, like Amazon EMR, Athena, SageMaker, or QuickSight

Thank You

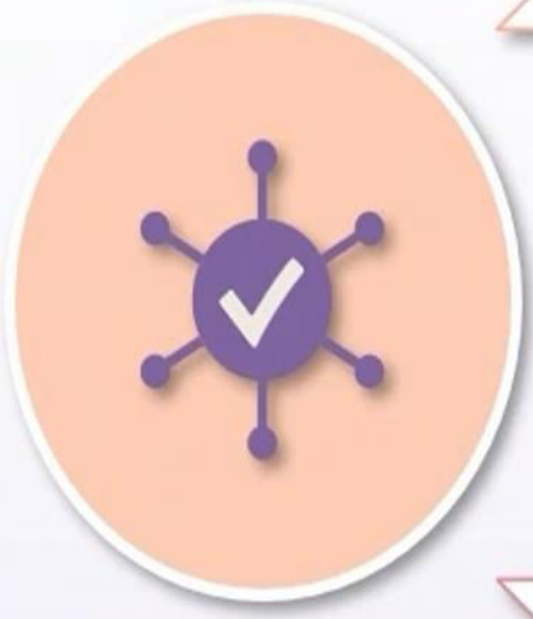
What Is Jupyter Notebook?

Garv Khurana

Jupyter NoteBook Advantages



Jupyter Notebook Features



- Easily customizable interfaces and extensions
- Used for data cleaning, transformations, creating statistical models, simulations, and visualizations
- Support Apache Spark, Python, R, and Scala

Jupyter Notebook Layout

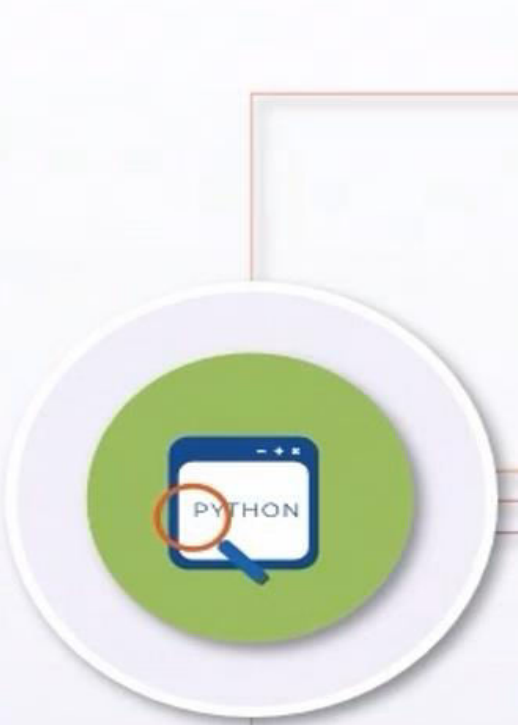
Executable computer code elements

Rich text elements, including images and links

Python for Data Science

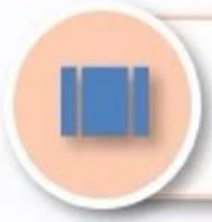
Garv Khurana

Python Features

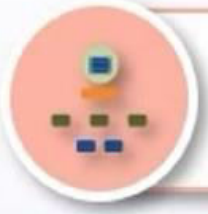


- Interpreted language
- Interactive and object-oriented
- Portable and cross-platform libraries
- Can be run on multiple platforms with the same interface
- Supports GUI applications

Python Packages



NumPy – fundamental computing and array functionality



Pandas – high performance data structures and data analysis tools

Matplotlib – visualization package for creating line plots, bar charts, histograms, etc.

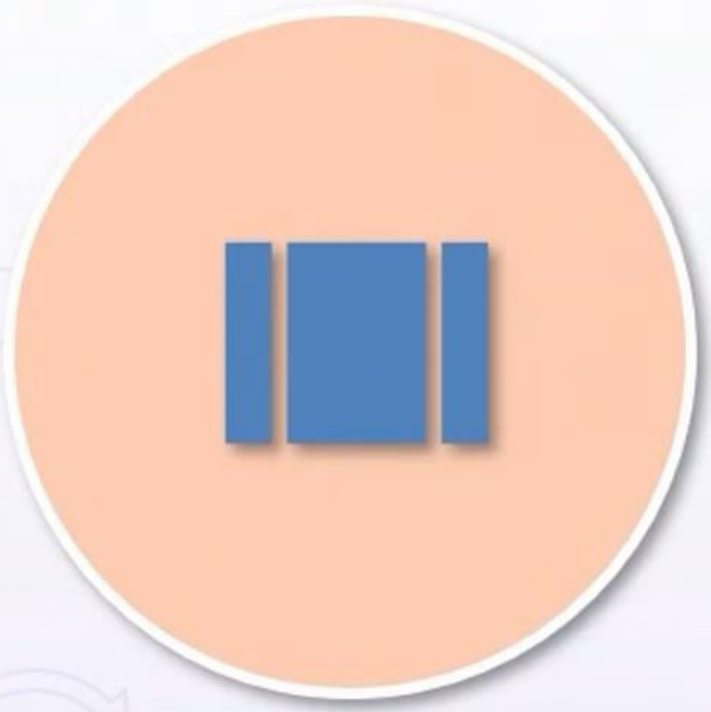
Seaborn – visualization package for creating high-level statistical graphics

scikit-learn – machine learning library

Python Packages - NumPy

Garv Khurana

NumPy Features



NumPy ndarray object

Fast array operations

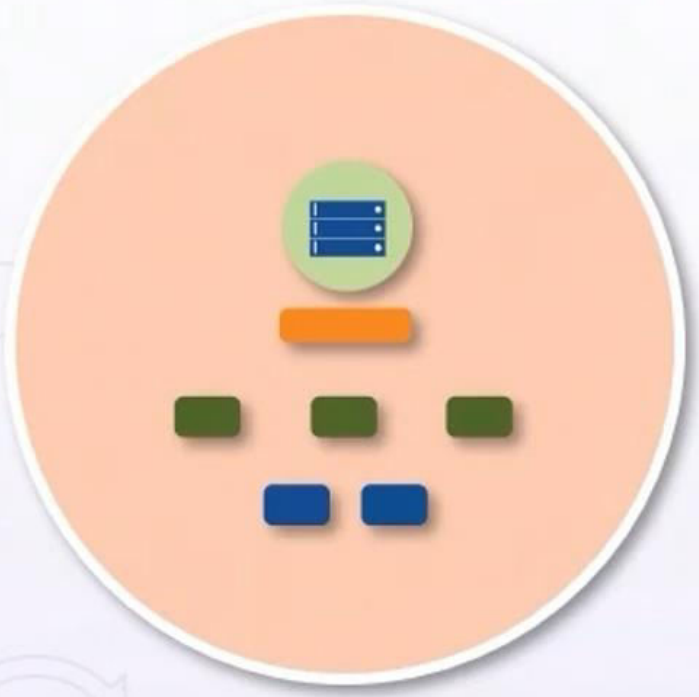
C/C++ optimization

Array operations are optimized to work with latest CPU architectures

Data type support

Integer, Boolean, float, datetime, and other data types available

Pandas Features



Pandas dataframe object

Data manipulation with integrated indexing

Data manipulation tools

Align, manage, filter, index, and slice datasets across various structures and formats

Import any data

Load datasets from CSV, JSON, SQL, and Excel files

Import Packages

In [ ]:

```
1 import pandas as pd
2 import numpy as np
```

Numpy [...]

Pandas [...]

Numpy

Applications:
1. Alternative to lists; better memory
2. Allows multidimensional arrays
3. Works with Pandas

In [3]:
```
# python array vs numpy array
python_array = [1,2,3,4,5]

numpy_array = np.array([1,2,3,4,5])

python_array
numpy_array
```

Out[3]: array([1, 2, 3, 4, 5])

In [4]:
```
# Check type of array
type(numpy_array), type(python_array)
```

Out[4]: (numpy.ndarray, list)

In [8]:
```
# Check dimension of an array
multi_dimension_array.ndim
```

Out[8]: 3

In [11]:
```
# Access one/two dimension of an array
multi_dimension_array[3][2]
```

Out[11]:
```
array([7, 0, 4, 1, 8, 6, 1, 5, 0, 2, 7, 7, 2, 5, 0, 0, 1, 2, 4, 3, 0, 4,
       8, 1, 3, 5, 3, 3, 8, 8, 3, 4, 3, 0, 2, 2, 3, 7, 0, 0, 6, 8, 0, 8,
       1, 3, 3, 7, 4, 4, 2, 6, 2, 3, 2, 8, 6, 6, 1, 5, 1, 3, 3, 4, 6, 2,
       5, 3, 6, 1, 3, 7, 3, 0, 8, 6, 2, 7, 7, 2, 1, 4, 7, 1, 6, 7, 1, 2,
       5, 5, 6, 4, 8, 1, 1, 0, 2, 4, 0, 7])
```

In [12]:
```
# summary statistics
multi_dimension_array[3][2].mean()
```

Out[12]: 3.68

Pandas

Read Data

From CSV: Demo

In [14]:
```
# upload
input_data = pd.read_csv("VIX.csv")
```

From SQL, MongoDB, or in other format such as JSON, Lists: Challenge [...]

Data Understanding + Summary Statistics

In [15]:
```
# Size of data frame
input_data.shape
```

Out[15]: (565, 7)

In [16]:
```
# head: Demo
input_data.head()
```

Out[16]:

	Date	Open	High	Low	Close	Adj Close	Volume
0	2018-04-20	16.160000	17.500000	15.19	16.879999	16.879999	0
1	2018-04-23	17.290001	17.559999	15.79	16.340000	16.340000	0
2	2018-04-24	16.160000	19.660000	15.37	18.020000	18.020000	0
3	2018-04-25	18.139999	19.840000	17.75	17.840000	17.840000	0
4	2018-04-26	18.070000	18.120001	16.24	16.240000	16.240000	0

In [ ]:
```
# tail
input_data.tail(6)
```

In [17]:
```
# summary statistics
input_data.describe()
```

Data Transformation

Cleaning

In [ ]:
```
# Looking for NA's
input_data.isnull().sum()
```

In [ ]:
```
# Viewing null column
input_data[input_data['Open'].isnull()]
```

In [ ]:
```
# Drop rows with NA
input_data.dropna(inplace=True)
```

In [20]:
```
# Dropping unneccessary columns
input_data.drop(['Volume'], axis = 1, inplace=True)
```

Adding features

In [ ]:
```
# Static
input_data['Type'] = 'VIX'
input_data.head(2)
```

In [ ]:
```
# Computation
input_data['Range'] = input_data['High'] - input_data['Low']
input_data.tail(2)
```

In [ ]:
```
# Rolling sum including current day
input_data['5_day_range_mean'] = input_data['Range'].rolling(5).mean()
input_data.head(15)
```

Python Packages - Matplotlib

Garv Khurana

Matplotlib Features

- Create simple visual graphics, like line plots and histograms
- Cartop tool allows creation of object-oriented map projection definitions
- Basemap tool allows creation of map projections, geopolitical boundaries, and coastlines

Seaborn Features

- Automatically perform statistical estimations on plotted variables
- Specialized plotting types for visualizing categorical data
- Quickly create combined composite plots
- Create complex figures linking dataset structure to a grid of axes

Using Matplotlib, Seaborn, & Bokeh for Data Analysis

Garv Khurana

Input Dataframe [...]

Matplotlib [...]

Seaborn [...]

In [25]:
```
1 input_data.head(15)
```

Out[25]:

	Date	Open	High	Low	Close	Adj Close	Type	Range	5_day_range_mean
0	2018-04-20	16.160000	17.500000	15.19	16.879999	16.879999	VIX	2.310000	NaN
1	2018-04-23	17.290001	17.559999	15.79	16.340000	16.340000	VIX	1.769999	NaN
2	2018-04-24	16.160000	19.660000	15.37	18.020000	18.020000	VIX	4.290000	NaN
3	2018-04-25	18.139999	19.840000	17.75	17.840000	17.840000	VIX	2.090000	NaN
4	2018-04-26	18.070000	18.120001	16.24	16.240000	16.240000	VIX	1.880001	2.468
5	2018-04-27	16.219999	16.770000	15.25	15.410000	15.410000	VIX	1.520000	2.310
6	2018-04-30	15.310000	16.350000	15.13	15.930000	15.930000	VIX	1.220000	2.200
7	2018-05-01	16.000000	16.820000	15.42	15.490000	15.490000	VIX	1.400000	1.622
8	2018-05-02	15.480000	15.970000	14.75	15.970000	15.970000	VIX	1.220000	1.448
9	2018-05-03	15.780000	18.660000	15.43	15.900000	15.900000	VIX	3.230000	1.718
10	2018-05-04	15.940000	16.920000	10.91	14.770000	14.770000	VIX	6.010000	2.616
11	2018-05-07	15.050000	15.270000	14.51	14.750000	14.750000	VIX	0.760000	2.524
		1.530000	15.560000	14.52	14.710000	14.710000	VIX	1.040000	2.452

Matplotlib

In [26]:
```
import matplotlib as mpl
import mplfinance as mpf

# Commands to install packages
# pip install matplotlib
# pip install mplfinance
```

Change the index of pandas dataframe to datetime

In [27]:
```
input_data.index = pd.to_datetime(input_data['Date'], format='%Y/%m/%d')
input_data.drop(['Date'], axis = 1, inplace=True)
input_data
```

Out[27]:

Date	Open	High	Low	Close	Adj Close	Type	Range	5_day_range_mean
2018-04-20	16.160000	17.500000	15.190000	16.879999	16.879999	VIX	2.310000	NaN
2018-04-23	17.290001	17.559999	15.790000	16.340000	16.340000	VIX	1.769999	NaN
2018-04-24	16.160000	19.660000	15.370000	18.020000	18.020000	VIX	4.290000	NaN
2018-04-25	18.139999	19.840000	17.750000	17.840000	17.840000	VIX	2.090000	NaN
2018-04-26	18.070000	18.120001	16.240000	16.240000	16.240000	VIX	1.880001	2.468

Plot simple candlestick chart

In [ ]:
```
mpf.plot(input_data, type='candle')
```

Plot candlestick chart by using a pre-existing style

In [ ]:
```
mpf.plot(input_data, type='candle', figratio = (12,7), style = 'yahoo')
```

Plot first 100 rows of the candlestick chart

In [ ]:
```
mpf.plot(input_data[:100], type='candle', figratio = (12,7), style = 'yahoo')
```

Seaborn

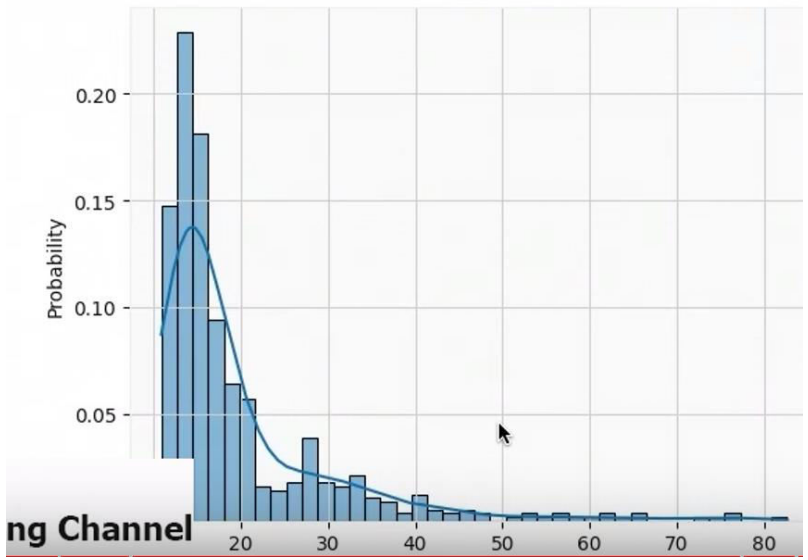
In [32]:
```
import seaborn as sns
import math
import matplotlib.pyplot as plt

# pip install seaborn
```

Frequency distribution of price

In [ ]:
```
sns.histplot(input_data.Close, kde=True)
```

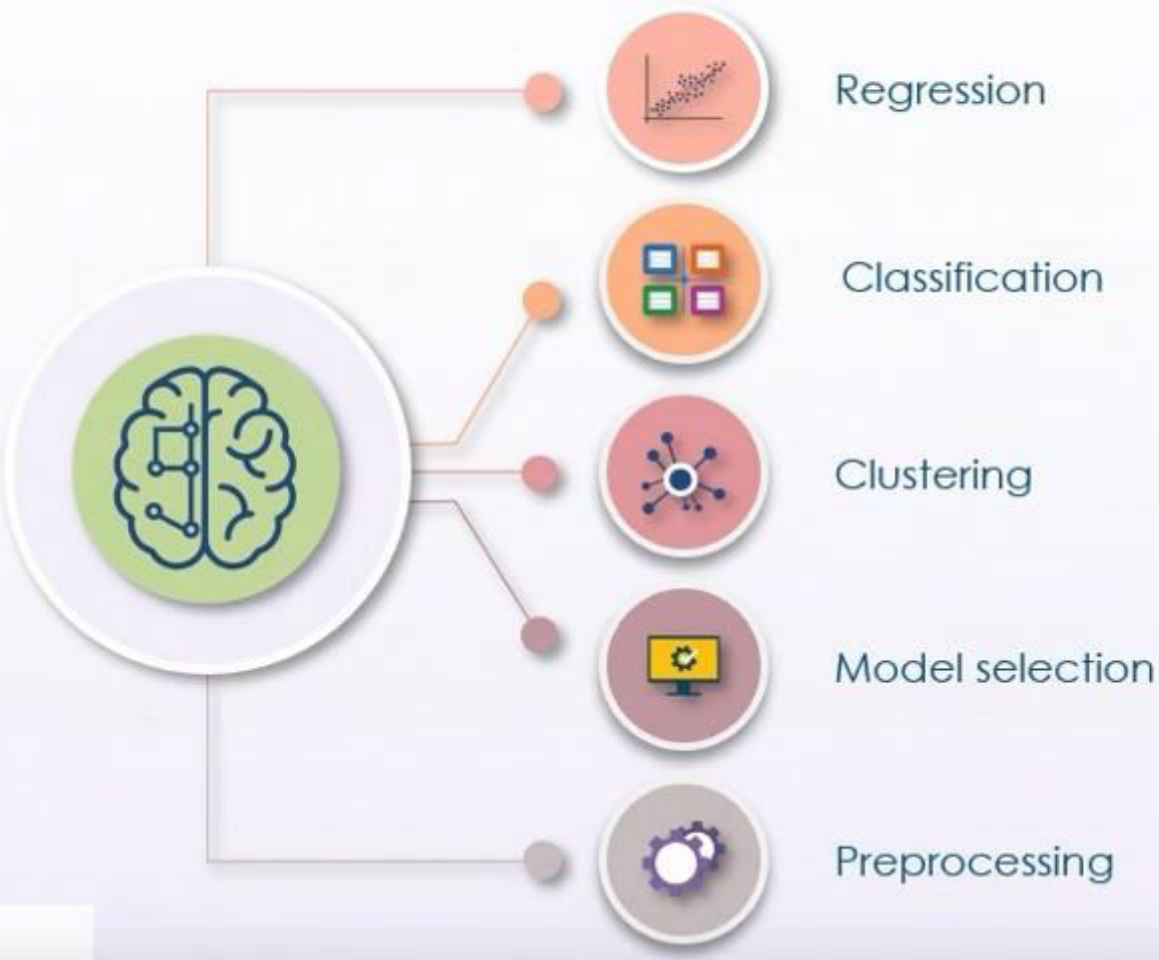
Percentage frequency distribution of price

In [ ]:
```
istplot(input_data.Close, kde=True, stat='probability')
```



Machine Learning in Python

Machine Learning with Scikit Learn



Plot: Leading digits of price data

In [ ]:
```
def get_first_digit(row):
    return int(str(row)[:1])

input_data['FirstDigit'] = input_data['Close'].apply(get_first_digit)
```

In [ ]:
```
input_data.head(6)
```

In [ ]:
```
sns.histplot(input_data.FirstDigit, kde=True, stat='probability')
```

Preprocessing data for ML

In [ ]:
```
from sklearn.preprocessing import MinMaxScaler
```

Converting data from pandas column

In [ ]:
```
prices = input_data.Close.values
prices = prices.reshape(-1,1)
prices
```

Normalize data between 1 and 100

In [35]:
```
scaler = MinMaxScaler(feature_range=(1,100))
normalised_prices = scaler.fit_transform(prices)
input_data['normalised_prices'] = normalised_prices
```

Get leading digits of normalised prices

In [ ]:
```
input_data['FirstDigitNormalised'] = input_data['normalised_prices'].apply(get_first_digit)
input_data.head()
```

Get leading digits of normalised prices

In [36]:
```
input_data['FirstDigitNormalised'] = input_data['normalised_prices'].apply(get_first_digit)
input_data.head()
```

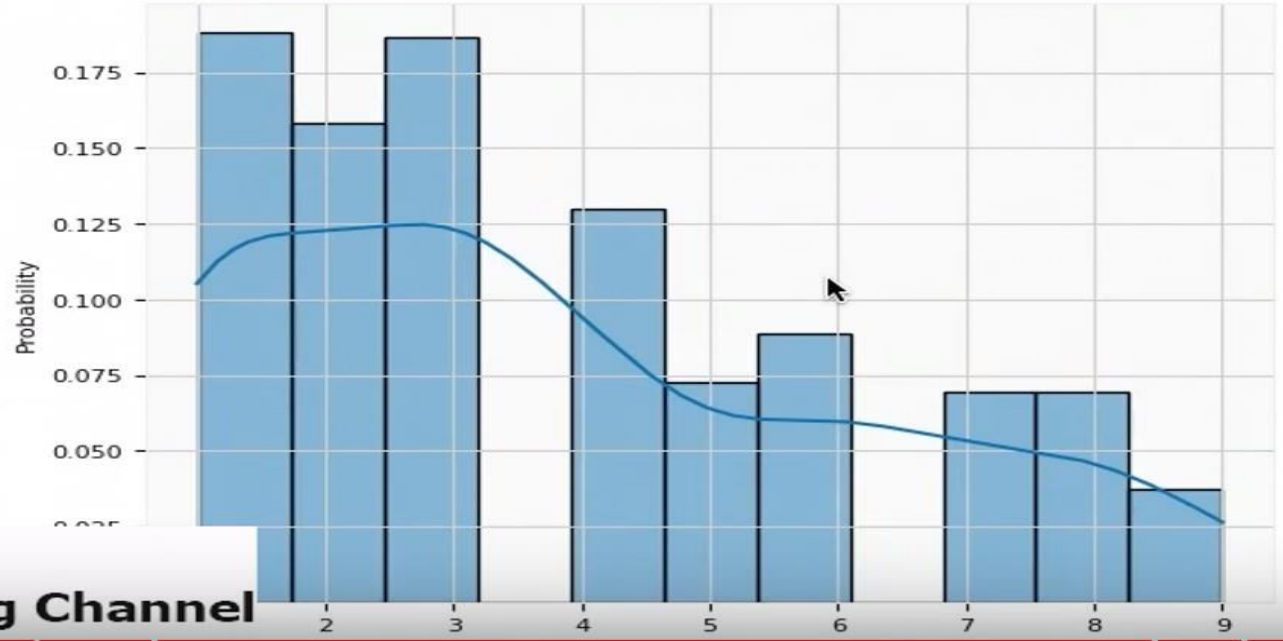
Out[36]:

Date	Open	High	Low	Close	Adj Close	Type	Range	5_day_range_mean	FirstDigit	normalised_prices	FirstDigitNormalised
2018-04-20	16.160000	17.500000	15.19	16.879999	16.879999	VIX	2.310000	NaN	1	9.309714	9
2018-04-23	17.290001	17.559999	15.79	16.340000	16.340000	VIX	1.769999	NaN	1	8.565562	8
2018-04-24	16.160000	19.660000	15.37	18.020000	18.020000	VIX	4.290000	NaN	1	10.880707	1
2018-04-25	18.139999	19.840000	17.75	17.840000	17.840000	VIX	2.090000	NaN	1	10.632656	1
2018-04-26	18.070000	18.120001	16.24	16.240000	16.240000	VIX	1.880001	2.468	1	8.427756	8

Plot: Leading digits of normalised price data

In [37]:
```
sns.histplot(input_data.FirstDigitNormalised, kde=True, stat='probability')
```

Out[37]: <AxesSubplot:xlabel='FirstDigitNormalised', ylabel='Probability'>



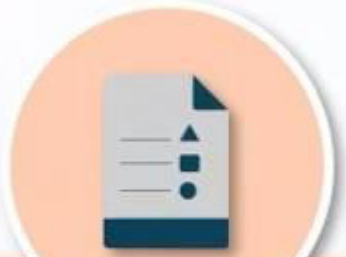
Thank you

Categorical and Numerical Data Types

Garv Khurana

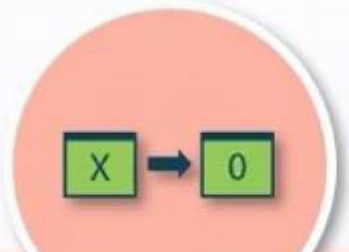
Data Types

Data Types



Categorical

- Qualitative data
- Deals with unquantifiable characteristics and descriptors
- Texture, color, flavor, smell



Numerical

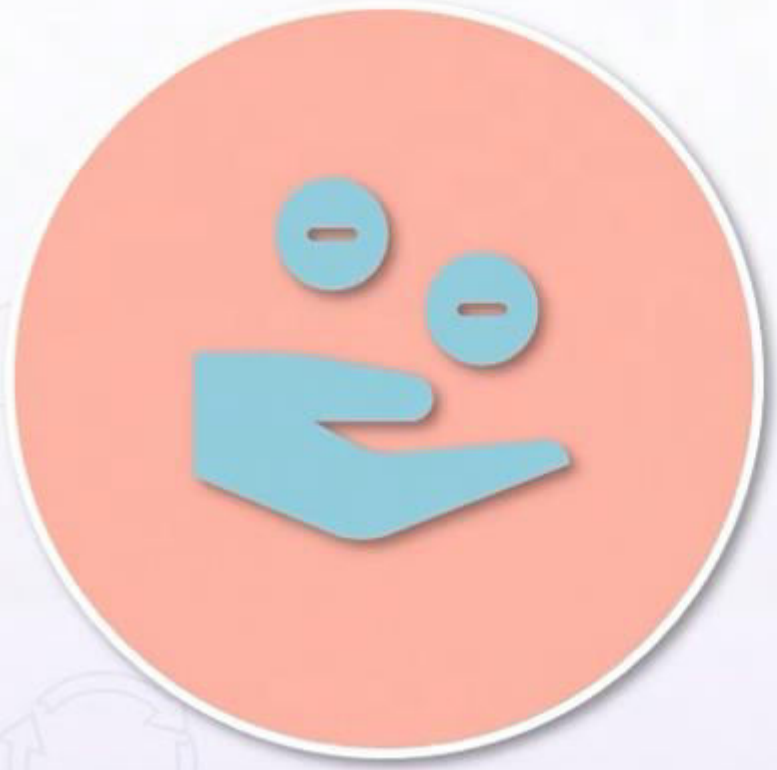
- Quantitative data
- Continuous or discrete
- Continuous data can be measured fractionally
- Discrete data consists of whole number measurements

Bernoulli, Uniform, and Binomial Data Distributions

Garv Khurana

Types of Data Distributions

Bernoulli Distribution



- Easiest data distribution available
- Two possible outcomes and single trial
- Mutually exclusive event probabilities

Uniform Distribution

- Derived from Bernoulli distribution
- Used for unlimited outcomes with exact same probability
- Roll of a dice follows uniform distribution
- Mutually exclusive event probabilities

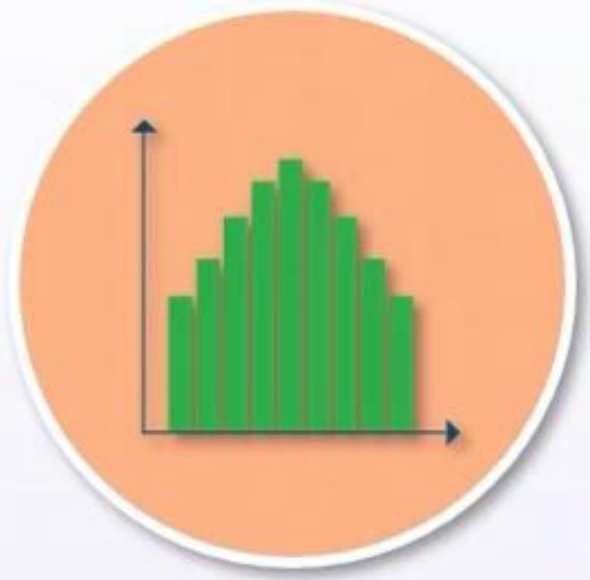
Binomial Distribution

- Sum of outcomes of an event that follows Bernoulli distribution
- Used for binary outcomes with same probability in each trial
- Goes through multiple successive independent trials

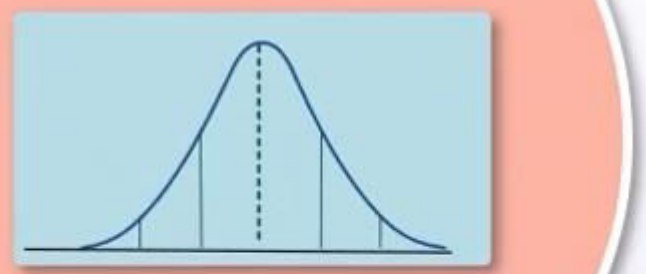
Normal, Poisson, and Exponential Data Distributions

Garv Khurana

Data Distribution Types Continued

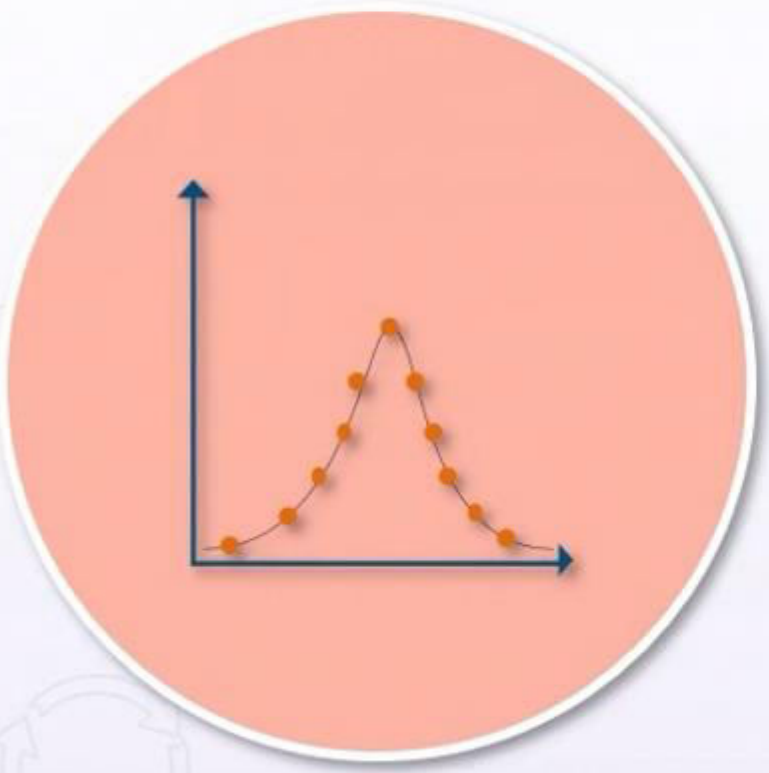


Normal (Gaussian) Distribution



- Most fundamental and commonly occurring data distribution
- Average height in a population, income distributions, or average grades by students typically follow Gaussian distribution
- Data is centered around mean values symmetrically

Poisson Distribution



- Discrete probability distribution for events occurring within certain time
- The average number of events per time period is known as a priori
- Exact moment when an event might occur is randomly spaced
- Events are independent of each other, but the average rate of occurrence is constant

Exponential Distribution

- Determines time taken between event occurrences
- Probability distribution of time passed between events in Poisson point process
- Estimates amount of time until a certain event will occur

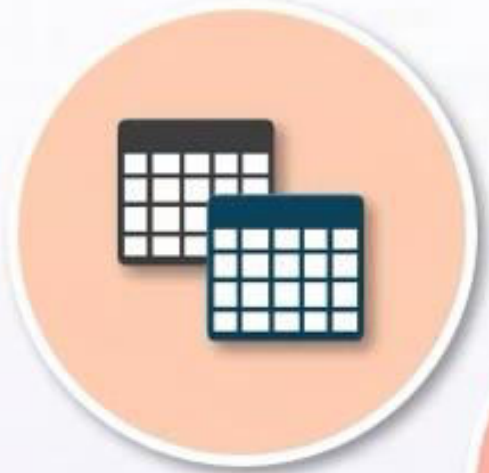
The Primary Role of Data Visualization

Garv Khurana

Data Visualizations

Data Visualization Types

Tables – quantitative data organized into rows and columns with attribute labels



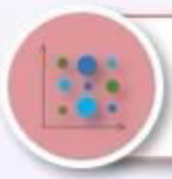
Graphs (charts) – graphical representation of data using line, bar, and point objects

Effective Visual Representation Characteristics

Show the data at hand

Encourage user to analyze the data

Avoid distracting the user with unnecessary information



Make large datasets visually coherent

Reveal micro and macro characteristics of datasets

Data Visualizations

Traditional graphic types

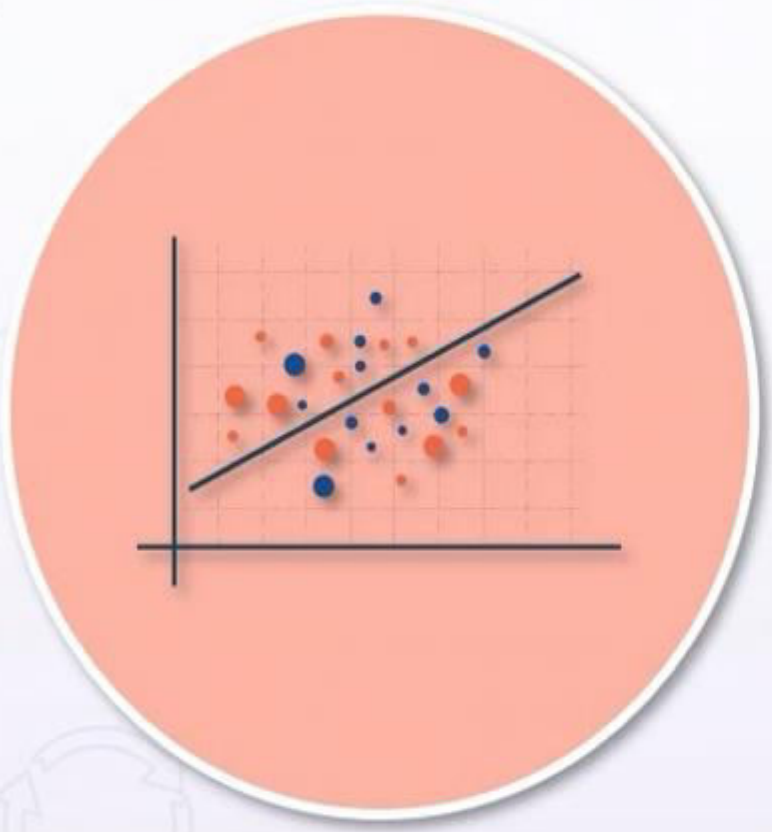
Line Charts

- Simple traditional charts
- Illustrate changes to a variable in relation to another variable
- Often illustrate changes over time

Bar Charts & Histograms

- Illustrate changes to a variable at various time points or compared to another variable
- Represent categorical data with bars proportional to values represented
- Histograms illustrate entire range of values over binned intervals

Scatter Plots



- Two or more variables are displayed in Cartesian coordinates
- Data points have associated X and Y values
- Different values can be differentiated using shape and color
- Can be extended to 3D visualizations

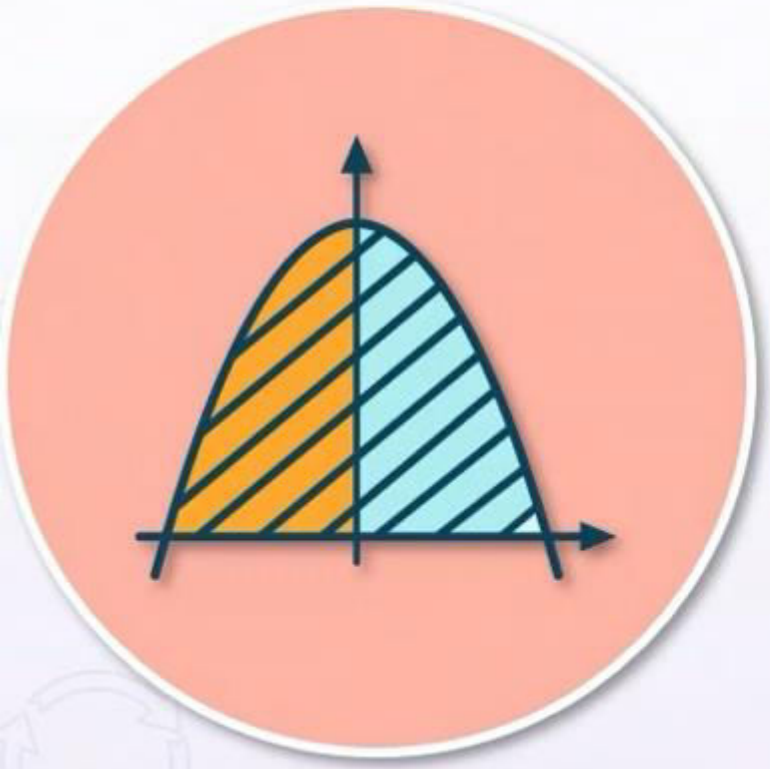
Data Visualization - Modern Graphic Types

Garv Khurana

Data Visualizations

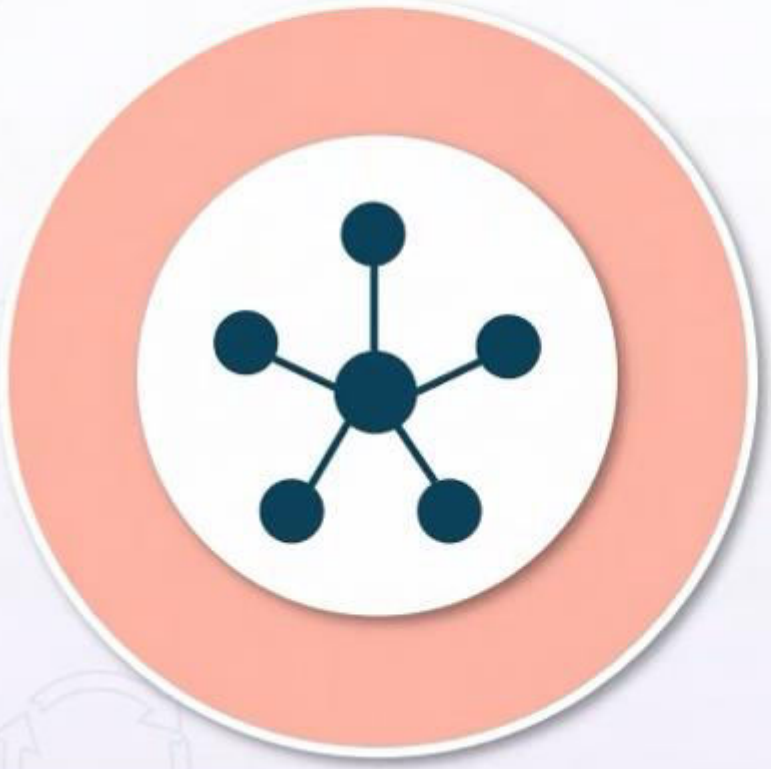
Modern graphic types

Stream Graphs



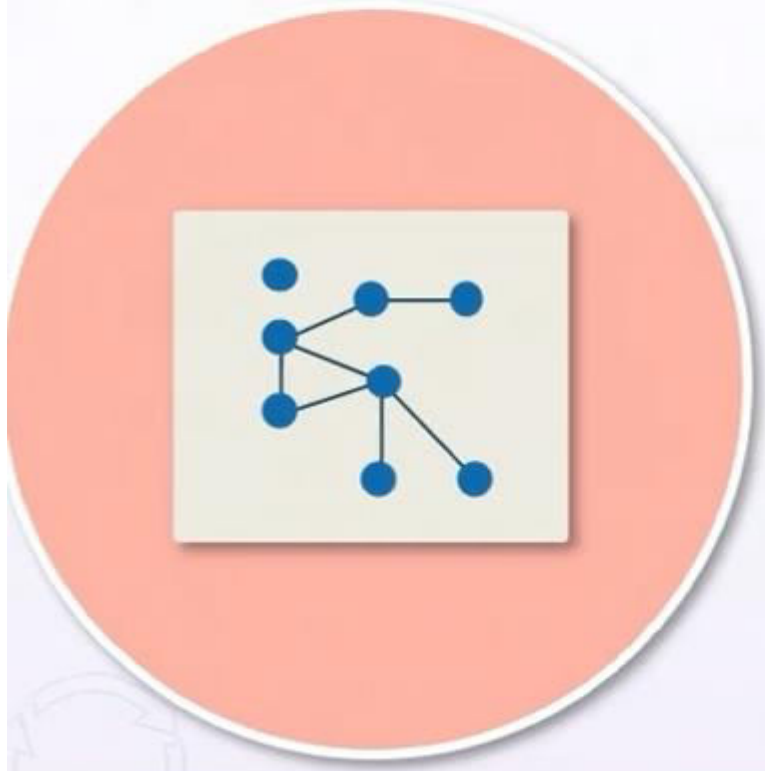
- Stacked area graphs displaced around central axes to form flowing shape
- Layers of the graph are positioned to minimize wiggle
- Can only be used to display data with positive values

Network Graphs



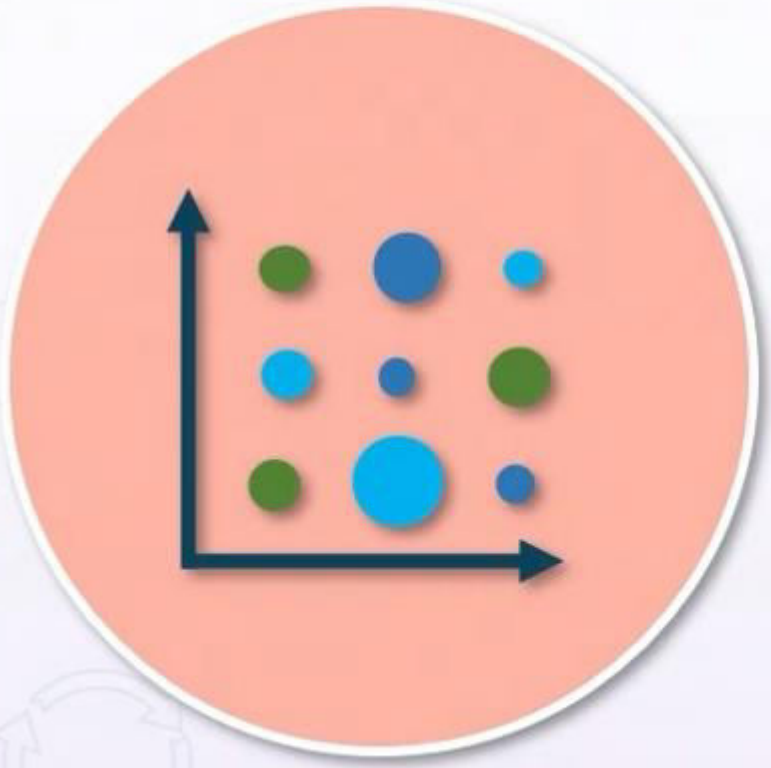
- Way of representing extremely large datasets with underlying clusters/groupings within a network
- Allow identification of most influential nodes
- Datasets can be distinguished by node color and thickness

Tree Maps



- Illustrate hierarchical data using nested rectangles
- Size and color of rectangles displays proportion and variable categories in a dataset

Heatmaps



- Represent phenomenon magnitude using color in two dimensions
- Color intensity variation gives visual representation of variable clusters in space
- Can be displayed in a form of matrix with fixed grid cells

Implementing Data Visualization with Python

Garv Khurana


Import Packages
[...]
Input Data to plot
[...]
Heatmap using matplotlib
[...]
Heatmap with Bokeh
[...]
Stacked times series plot
[...]

Import Packages

In [2]:

```
# bokeh
from bokeh.io import output_notebook

from bokeh.io import output_file, show
from bokeh.models import (BasicTicker, ColorBar, ColumnDataSource,
                          LinearColorMapper, PrintfTickFormatter,)
from bokeh.plotting import figure
from bokeh.sampledata.unemployment1948 import data
from bokeh.transform import transform
from bokeh.palettes import Spectral4
from bokeh.plotting import figure, output_file, show
from bokeh.sampledata.stocks import AAPL, GOOG, IBM, MSFT


# pandas
import pandas as pd
from pandas import read_csv
from pandas import DataFrame
from pandas import Grouper

# matplotlib
from matplotlib import pyplot

output_notebook()
```

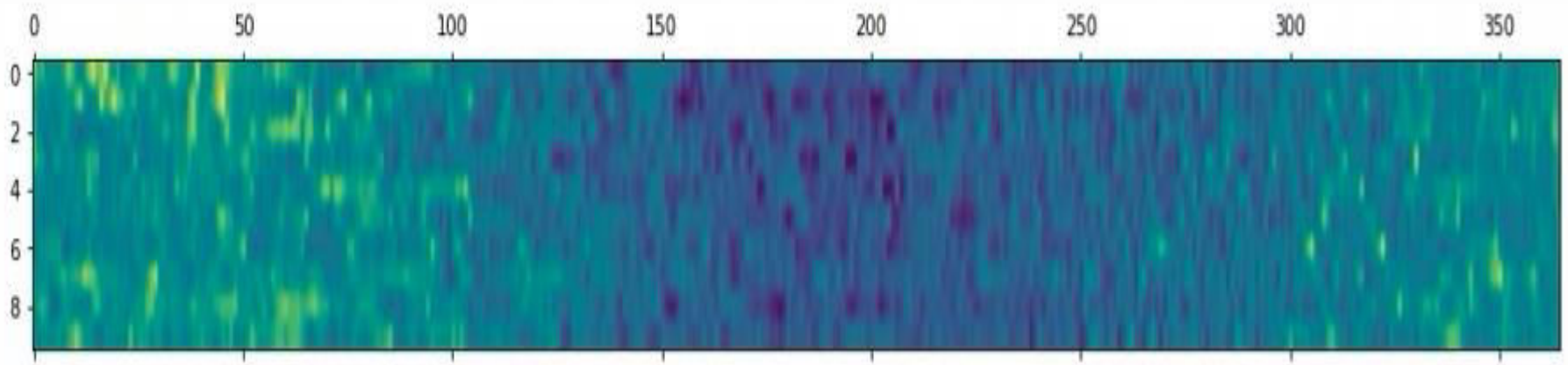
Input Data to plot

In [ ]:
```
series = read_csv('daily-min-temperatures.csv', header=0, index_col=0, parse_dates=True, squeeze=True)
series
```

Out[3]:
```
Date
1981-01-01    20.7
1981-01-02    17.9
1981-01-03    18.8
1981-01-04    14.6
1981-01-05    15.8
              ...
1990-12-27    14.0
1990-12-28    13.6
1990-12-29    13.5
1990-12-30    15.7
1990-12-31    13.0
Name: Temp, Length: 3650, dtype: float64
```

Heatmap using matplotlib

In [4]:
```
# plot
groups = series.groupby(Grouper(freq='A'))
years = DataFrame()
for name, group in groups:↔
years = years.T

pyplot.matshow(years, interpolation=None, aspect='auto')
pyplot.show()
```



Heatmap with Bokeh

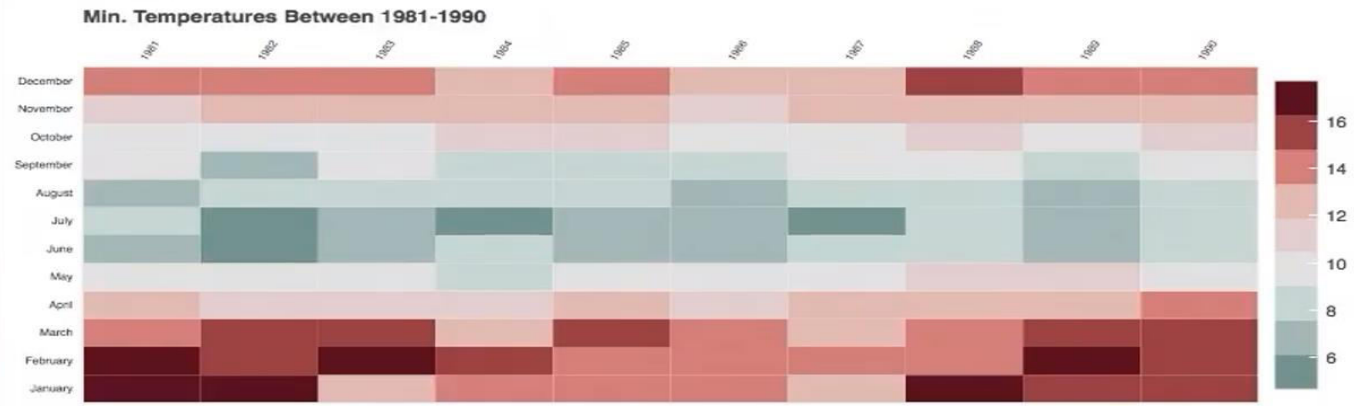
In [5]:
```
# data preparation step↔
```

In [ ]:
```
# plot

source = ColumnDataSource(df)

# this is the colormap from the original NYTimes plot
colors = ["#75968f", "#a5bab7", "#c9d9d3", "#e2e2e2", "#dfccce", "#ddb7b1", "#cc7878", "#933b41", "#550b1d"]
mapper = LinearColorMapper(palette=colors, low=df.rate.min(), high=df.rate.max())

p = figure(plot_width=800, plot_height=300, title="Min. Temperatures Between 1981-1990",
           x_range=years, y_range=list(df.Month.unique()),
           toolbar_location=None, tools="", x_axis_location="above")

p.rect(x="Year", y="Month", width=1, height=1, source=source,
       line_color=None, fill_color=transform('rate', mapper))

color_bar = ColorBar(color_mapper=mapper,
                     ticker=BasicTicker(desired_num_ticks=len(colors)),)

p.add_layout(color_bar, 'right')

p.axis.axis_line_color = None
ajor_tick_line_color = None
ajor_label_text_font_size = "7px"
ajor_label_standoff = 0
major_label_orientation = 1.0
```

Learning Channel

In [5]:
```
# data preparation step↔
```

In [6]:
```
# plot↔
```

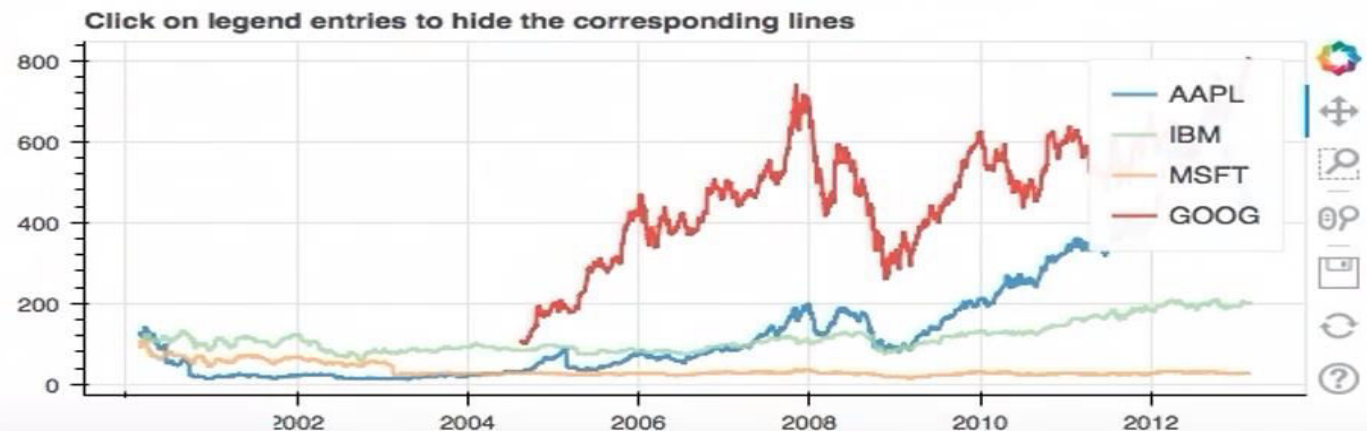


Stacked times series plot

In [8]:
```
# 1. PRPEARE PLOT
p = figure(plot_width=600, plot_height=250, x_axis_type="datetime")
p.title.text = 'Click on legend entries to hide the corresponding lines'


show(p)
```

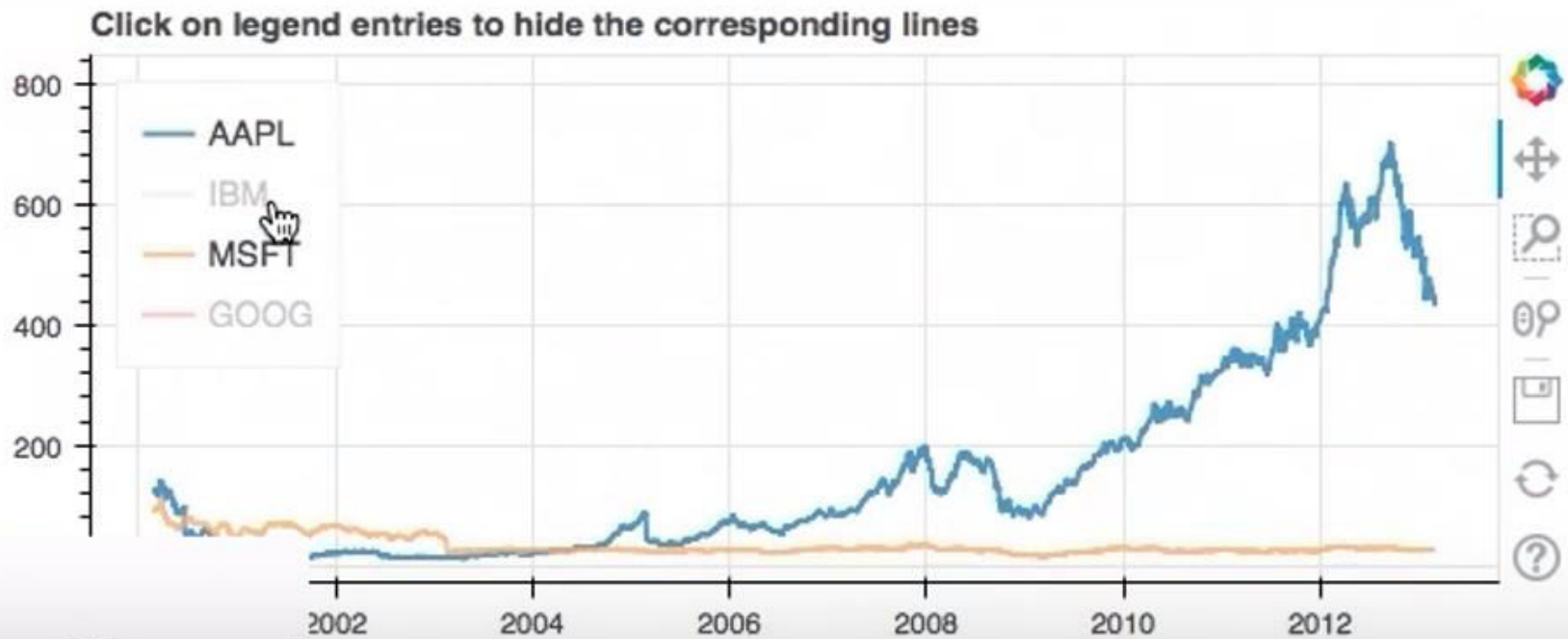
```
WARNING:bokeh.core.validation.check:W-1000 (MISSING_RENDERERS): Plot has no renderers: Figure(id='1296', ...)
```

In [9]:
```
# 2nd Step: PREAPRE AND ADD DATA TO PLOT
for data, name, color in zip([AAPL, IBM, MSFT, GOOG], ["AAPL", "IBM", "MSFT", "GOOG"], Spectral4):
    df = pd.DataFrame(data)
    df['date'] = pd.to_datetime(df['date'])
    p.line(df['date'], df['close'], line_width=2, color=color, alpha=0.8, legend_label=name)

show(p)

```



In [10]:

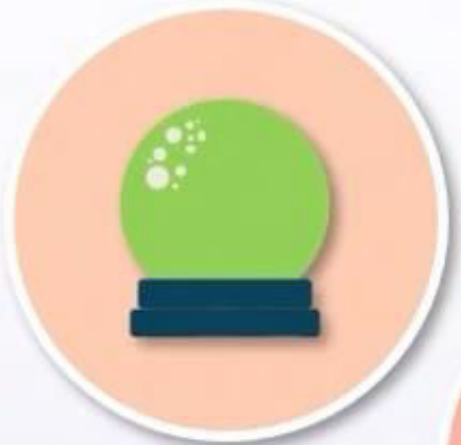
```
1  # PREPARE PLOT LAYOUT AND FEATURES
2  p.legend.location = "top_left"
3  p.legend.click_policy="hide"
4
5  show(p)
```

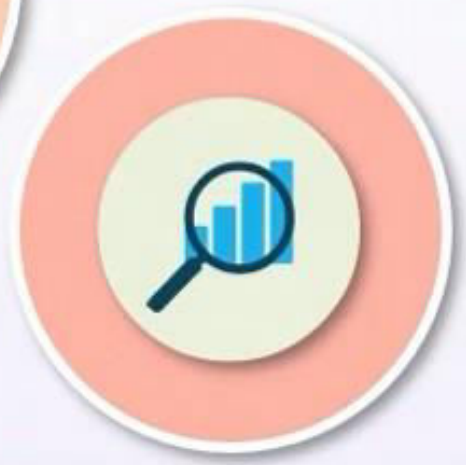


Time Series Analysis

Time Series Models

Future value predictions





Plotted using line charts with trend lines and moving averages

Time Series Data

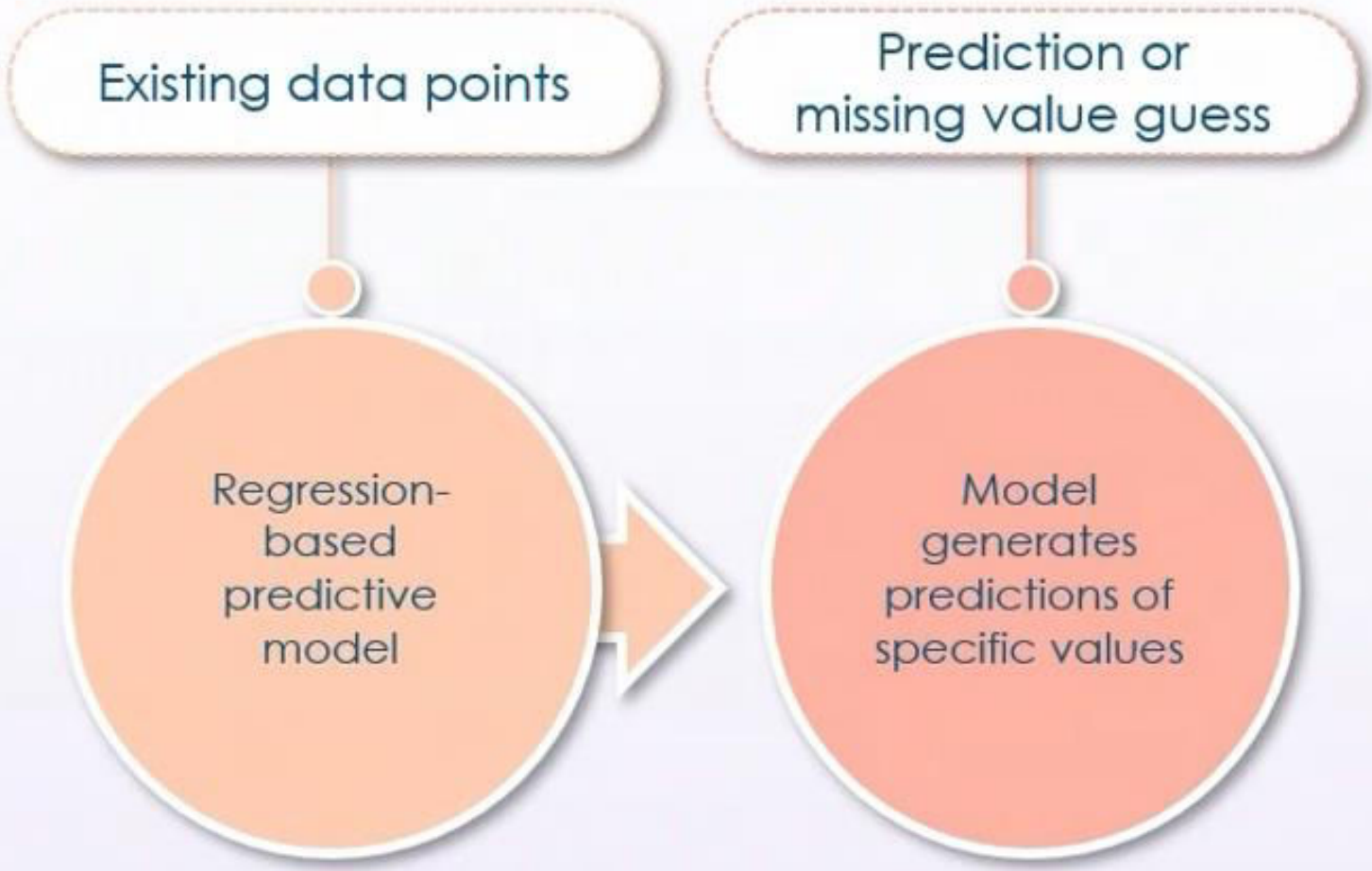
Time series – data points are collected at successive time periods

Cross sectional data – data points are analyzed at a single point in time

Time Series Use

- Economic and financial domains
- Social sciences
- Medical data collection and tracking
- Geophysical monitoring of temperature and pollution

Time Series Forecasting



Time Series Analysis

Trends, seasonality,
autocorrelation

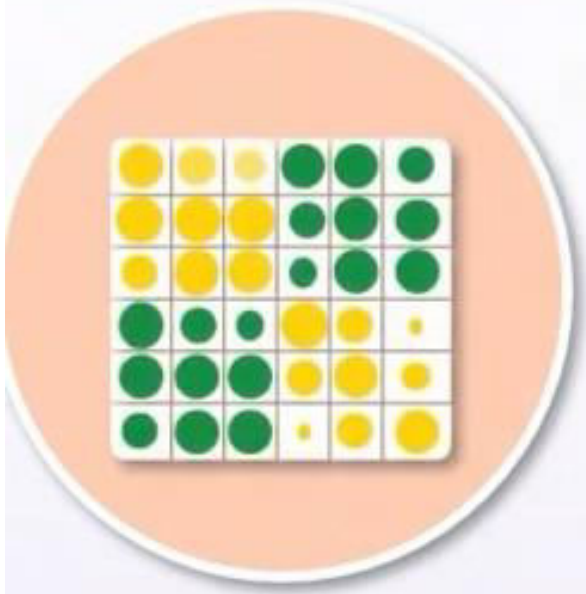
Time Series Data Trends

Values of a variable are proportionate to the time period in an increasing/decreasing pattern or centered around time-independent mean

Time Series Data Seasonality

Data exhibits regular and predictable patterns at short time-period intervals

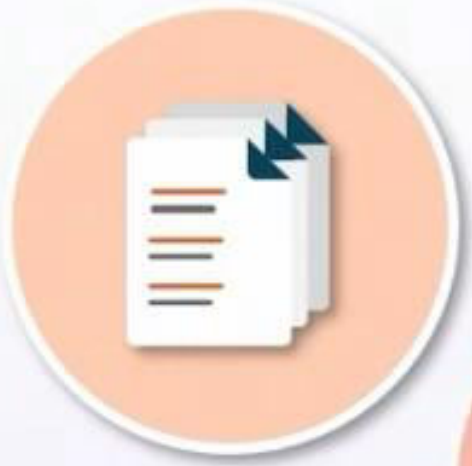
Time Series Data Autocorrelation

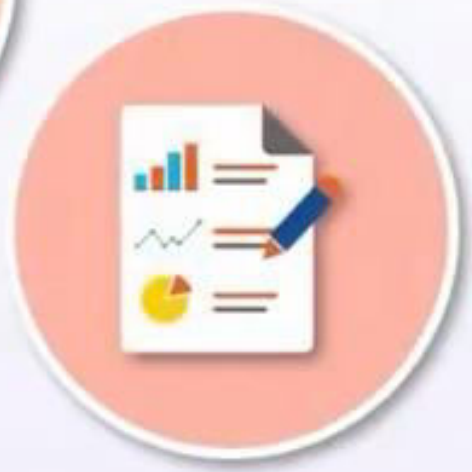


The degree of correlation between the observations of the same dataset at different points in time

Time Series Data Visualization

Overlapping charts





Separated charts

Import Packages

In [1]:

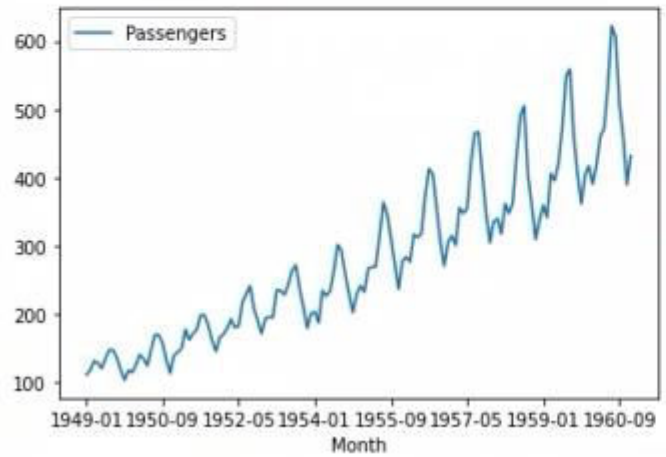
```
# pandas
from pandas import read_csv, to_datetime

# matplotlib
from matplotlib import pyplot

# statsmodels
from statsmodels.tsa.seasonal import seasonal_decompose
```

Import and Plot Data [...]

Analysing Trend and Seasonality [...]

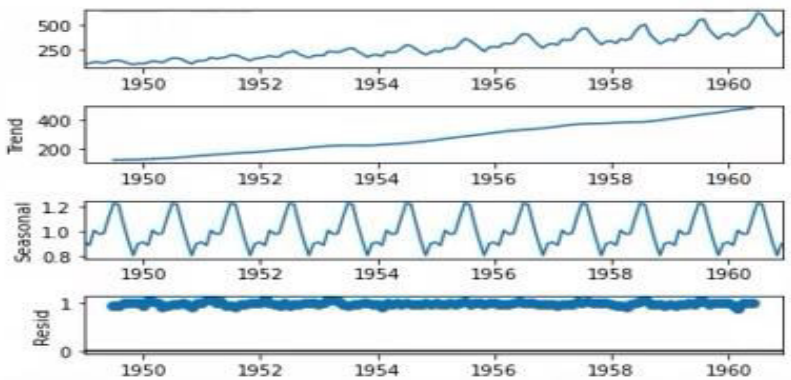
Import and Plot Data

In [2]:
```
series = read_csv('airline-passengers.csv', header=0, index_col=0)
series.plot()
pyplot.show()
```



Analysing Trend and Seasonality

In [3]:
```
#
series.reset_index(inplace=True)
series.index = to_datetime(series['Month'], format='%Y-%m', errors='coerce')
series.drop(['Month'], axis=1, inplace=True)

result = seasonal_decompose(series, model='multiplicative')
result.plot()
pyplot.show()
```



- Data types and distributions
- Traditional and modern data visualizations
- Time series analysis

Data Analysis Fundamentals

THANK YOU